

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação.

Orientadores: Diego Leonel Cadette Dutra
Cláudio Luís de Amorim

Rio de Janeiro
Agosto de 2026

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

DISSERTAÇÃO SUBMETIDA AO CORPO DOCENTE DO INSTITUTO ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU DE MESTRE EM CIÊNCIAS EM ENGENHARIA DE SISTEMAS E COMPUTAÇÃO.

Orientadores: Diego Leonel Cadette Dutra
Cláudio Luís de Amorim

Aprovada por: Prof. Diego Leonel Cadette Dutra
Prof. Nelson Maculan Filho
Dr. Mario Veiga Ferraz Pereira

RIO DE JANEIRO, RJ – BRASIL
AGOSTO DE 2026

Cruz, Marcelo Barros da

Investigando Problemas de Escalabilidade de E/S no Algoritmo SDDP em Ambientes HPC/Marcelo Barros da Cruz. – Rio de Janeiro: UFRJ/COPPE, 2026.

XIV, 65 p.: il.; 29, 7cm.

Orientadores: Diego Leonel Cadette Dutra

Cláudio Luís de Amorim

Dissertação (mestrado) – UFRJ/COPPE/Programa de Engenharia de Sistemas e Computação, 2026.

Referências Bibliográficas: p. 62 – 65.

1. MPI-IO. 2. Lustre. 3. SDDP. 4. Computação de Alto Desempenho. 5. Escalabilidade de E/S. I. Dutra, Diego Leonel Cadette *et al.* II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia de Sistemas e Computação. III. Título.

Dedico este trabalho à minha esposa e à minha filha, pela paciência e compreensão com o tempo que dediquei à pesquisa nos finais de semana.

Agradecimentos

Agradeço à PSR por tornar possível a realização desta pesquisa. O acesso ao modelo comercial SDDP, à infraestrutura computacional de alto desempenho e aos dados de estudos reais permitiu investigar os gargalos de E/S e avaliar as otimizações propostas em condições representativas de uso industrial — algo que dificilmente seria viável fora de um ambiente como o da empresa.

Aos professores Diego Dutra e Cláudio Amorim, agradeço por todos os ensinamentos e pela orientação ao longo destes três anos de curso. A dedicação, a paciência e o rigor com que conduziram este trabalho foram fundamentais para o meu amadurecimento como pesquisador e para que esta dissertação chegasse à sua forma final.

Aos companheiros de trabalho Luiz Carlos, Silvio Binato e Luiz Raphael, agradeço pela generosidade e pela disponibilidade em esclarecer minhas inúmeras dúvidas sobre o funcionamento do SDDP. As muitas conversas técnicas foram decisivas para a compreensão do modelo e para o direcionamento das análises apresentadas neste trabalho.

Resumo da Dissertação apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Mestre em Ciências (M.Sc.)

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Agosto/2026

Orientadores: Diego Leonel Cadette Dutra
Cláudio Luís de Amorim

Programa: Engenharia de Sistemas e Computação

A escalabilidade eficiente de E/S continua sendo um desafio crítico em aplicações paralelas de dados massivos, especialmente em sistemas de alto desempenho que lidam com grandes volumes de dados. Embora avanços como o uso de MPI-IO e sistemas de arquivos paralelos como o Lustre tenham contribuído significativamente, muitas aplicações ainda enfrentam gargalos que comprometem a performance. Esse problema é particularmente evidente em modelos de otimização utilizados no setor energético, como o SDDP, que demandam acesso frequente e distribuído a grandes conjuntos de dados. A eficiência da E/S torna-se, assim, um fator decisivo para a viabilidade e o desempenho de simulações complexas em ambientes paralelos. Esta dissertação investiga os gargalos de E/S da fase de simulação final do algoritmo SDDP e propõe uma arquitetura baseada em MPI-IO sobre Lustre, avaliada empiricamente nos ambientes AWS (FSx for Lustre) e Santos Dumont (LNCC).

Abstract of Dissertation presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Master of Science (M.Sc.)

INVESTIGATING I/O SCALABILITY ISSUES IN THE SDDP ALGORITHM ON HPC ENVIRONMENTS

Marcelo Barros da Cruz

August/2026

Advisors: Diego Leonel Cadette Dutra
Cláudio Luís de Amorim

Department: Systems Engineering and Computer Science

Efficient I/O scalability remains a major challenge for parallel applications handling massive data volumes, particularly in high-performance computing environments. Although advances such as MPI-IO and parallel file systems like Lustre have significantly improved performance, many applications still face I/O bottlenecks that hinder scalability. This issue is particularly critical in energy sector optimization models, such as Stochastic Dual Dynamic Programming (SDDP), which require frequent and distributed access to large datasets. Therefore, optimizing I/O operations becomes essential to ensure the feasibility and efficiency of complex simulations in parallel computing systems. This dissertation investigates the I/O bottlenecks of the final simulation phase of the SDDP algorithm and proposes an MPI-IO-based architecture over Lustre, empirically evaluated on AWS (FSx for Lustre) and on the Santos Dumont supercomputer (LNCC).

Sumário

Lista de Figuras **x**

Lista de Tabelas **xii**

Lista de Abreviaturas **xiv**

1 Introdução **1**

1.1 Contexto 1
1.2 Motivação 2
1.3 Objetivos e Contribuições 3
1.4 Organização 4

2 Fundamentação Teórica **5**

2.1 MPI, protocolos de comunicação e operações paralelas de arquivos . . 5
2.2 Sistema de arquivos paralelo Lustre 10
2.3 Ambientes computacionais 12
2.3.1 Ambiente AWS 13
2.3.2 Ambiente de supercomputação: Santos Dumont 14
2.4 O Programa Mensal da Operação (PMO) 14
2.5 Algoritmo SDDP 15
2.5.1 Comparação dos formatos dos resultados 16
2.5.2 Arquitetura atual de E/S do SDDP 17

3 Paralelização dos Mecanismos de E/S do SDDP **19**

3.1 Visão geral da arquitetura 19
3.2 Modelo de execução 20
3.3 Detalhes de implementação 22
3.3.1 Implementação das escritas independentes 22
3.3.2 Formato dos resultados 26
3.4 Instrumentação 28
3.4.1 Registros gerados pela instrumentação 29
3.4.2 Tempo de computação 30

3.4.3 Tempo de `MPI_File_open`/`MPI_File_close` 30
3.4.4 Tempo total de E/S 30
3.4.5 Tempo de bloqueio na transferência dos blocos de dados . . . 31
3.4.6 Banda agregada média percebida pela aplicação 31

4 Avaliações **33**
4.1 Experimento AWS 35
4.1.1 Avaliação da implementação atual 36
4.1.2 Avaliação da implementação MPI-IO com escritas independentes 37
4.1.3 Análise comparativa 38
4.1.4 Avaliação do impacto do número de OSTs 43
4.2 Experimento SDumont 46
4.2.1 Avaliação da implementação atual 47
4.2.2 Avaliação da implementação MPI-IO com escritas independentes 48
4.2.3 Análise comparativa 48
4.3 Síntese das avaliações 54

5 Trabalhos relacionados **56**

6 Conclusões e trabalhos futuros **58**

Referências Bibliográficas **62**

Lista de Figuras

2.1 Comparação esquemática entre os protocolos *Eager* e *Rendezvous* em comunicação MPI ponto a ponto. No *Eager*, o emissor transmite os dados imediatamente, sem negociação prévia; no *Rendezvous*, há uma etapa de *handshake* entre emissor e receptor antes da transferência efetiva, eliminando a necessidade de *buffers* intermediários. 6
2.2 Relação entre comunicação ponto a ponto e E/S paralela. No modelo centralizado, os resultados passam por um processo escritor; no modelo MPI-IO, os próprios processos escrevem diretamente no arquivo. 7
2.3 Diferença entre escritas independentes e coletivas em MPI-IO. Escritas independentes preservam autonomia dos processos; escritas coletivas permitem agregação e reordenação das requisições pela biblioteca MPI-IO. 9
2.4 Arquitetura do sistema de arquivos paralelo Lustre. O servidor de metadados (MDS/MDT) mantém o *namespace*, enquanto o Lustre distribui os dados entre os OSTs gerenciados pelos OSSs. Os clientes consultam o MDS para localizar o arquivo e, em seguida, acessam os dados diretamente nos OSTs. 11
2.5 Distribuição de um arquivo em *stripes* pelos OSTs no Lustre. Com *stripe count* igual a 4, o Lustre aloca os blocos de forma circular entre os OSTs, viabilizando o acesso paralelo aos dados. 12
2.6 Estrutura de cenários e estágios do SDDP. Cada célula corresponde a um subproblema $(\text{estágio}, \text{cenário})$. Os cenários são mutuamente independentes e podem ser distribuídos entre *ranks* MPI, caracterizando uma aplicação *bag-of-tasks*. 16
2.7 Comparação esquemática entre os formatos de resultados por patamares e horário do SDDP. 17
2.8 Arquitetura atual de E/S do SDDP com o *rank* 0 atuando como processo distribuidor e o *rank* 1 como processo escritor. 17

3.1 Arquitetura proposta com MPI-IO e Lustre, com o *rank* 0 atuando como processo distribuidor e os demais processos escrevendo diretamente em arquivos compartilhados. 20
3.2 Linha de tempo da simulação final do SDDP com MPI-IO. O *rank* 0 atua apenas como processo distribuidor; os *ranks* trabalhadores executam cenários, sincronizam na chamada coletiva `MPI_File_open` antes da primeira escrita, realizam escritas independentes e sincronizam novamente em `MPI_File_close`. 22
3.3 Fluxo de decisão para escritas independentes com MPI-IO (limite de 128 KB adotado neste trabalho como parâmetro de projeto, conforme discussão no texto). 23
3.4 Formato lógico dos arquivos binários de resultado do SDDP. 27

4.1 Distribuição agregada do número de escritas por faixa de tamanho de bloco no experimento (frequência em escala logarítmica; linha: percentual do total). 34
4.2 Volume total de dados gravado por faixa de tamanho de bloco no experimento (barras em GB; linha: número de escritas em escala logarítmica). 35
4.3 Banda agregada média percebida pela aplicação no Experimento AWS. 40
4.4 Tempo de execução no Experimento AWS. 42
4.5 Tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento AWS (escala logarítmica). 43
4.6 Tempo médio de envio por classe de tamanho das mensagens para configurações MPI-IO com 4 e 10 OSTs. 45
4.7 Banda agregada média percebida pela aplicação na implementação MPI-IO para configurações com 4 e 10 OSTs em 16 e 32 nós. 46
4.8 Banda agregada média percebida pela aplicação nas implementações atual e MPI-IO no Experimento SDumont. 50
4.9 Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont. 51
4.10 Tempo médio de bloqueio das operações de envio por tamanho de bloco de dados nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica). 53

Lista de Tabelas

2.1 Critérios práticos para escolha do modo de escrita em MPI-IO. 10

3.1 Arquivos de *log* gerados pela instrumentação, por *rank p*. 29

4.1 Desempenho do Experimento AWS com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, *speedup*, eficiência e percentual de I/O. Este trabalho calcula a eficiência em relação à base de 2 nós. 37

4.2 Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, coordenação, tempo total, *speedup*, eficiência e percentual de I/O. Este trabalho calcula a eficiência em relação à base de 2 nós. 38

4.3 Resumo comparativo entre a implementação atual e MPI-IO no Experimento AWS. Este trabalho calcula a melhoria no total e a melhoria de I/O em relação à implementação atual; valores negativos indicam piora. 38

4.4 Desempenho do Experimento SDumont com a implementação atual. O tempo total corresponde ao tempo de execução medido, que inclui computação, E/S e comunicação. Este trabalho calcula a eficiência em relação à base de 2 nós. 47

4.5 Desempenho do Experimento SDumont com a implementação MPI-IO. O tempo total corresponde ao tempo de execução medido, que inclui computação, E/S, comunicação e `MPI_File_open`/`MPI_File_close`. Este trabalho calcula a eficiência em relação à base de 2 nós. . 48

4.6 Resumo comparativo entre a implementação atual e MPI-IO no Experimento SDumont. Este trabalho calcula a melhoria no total e a melhoria de I/O em relação à implementação atual; valores negativos indicam piora. 49

Lista de Códigos

3.1 Pseudocódigo da escrita independente executada por cada *rank* trabalhador ao concluir um cenário (método `escreveRegistro`), anotado segundo os aspectos de acesso a dados do MPI-IO — *positioning*, *coordination* e *synchronism* [1, 2] — e a decisão de *layout* dos blocos. 26

Lista de Abreviaturas

AWS	Amazon Web Services 3
E/S	Entrada/Saída 1
EFA	Elastic Fabric Adapter 3
FSx	Amazon FSx for Lustre 3
HPC	High-Performance Computing 1
LNCC	Laboratório Nacional de Computação Científica 3
MDS	Metadata Server 3
MDT	Metadata Target 3
MPI	Message Passing Interface 1
MPI-IO	MPI Input/Output 3
ONS	Operador Nacional do Sistema Elétrico 3
OSS	Object Storage Server 3
OST	Object Storage Target 3
PMO	Programa Mensal da Operação 3
RDMA	Remote Direct Memory Access 3
SDDP	Stochastic Dual Dynamic Programming 1
SIN	Sistema Interligado Nacional 3
SINAPAD	Sistema Nacional de Processamento de Alto Desempenho . . . 3

Capítulo 1

Introdução

1.1 Contexto

A computação de alto desempenho (do inglês *High-Performance Computing* — HPC) tornou-se ferramenta indispensável para aplicações científicas e de engenharia que processam volumes crescentes de dados [3]. Em ambientes HPC modernos, o desempenho global de uma aplicação não depende apenas da capacidade computacional dos processadores, mas também da eficiência das operações de entrada e saída (E/S), que frequentemente se constituem como um dos principais gargalos da execução paralela em larga escala [4, 5]. À medida que o número de nós computacionais cresce, requisições concorrentes ao sistema de arquivos, contenção sobre adaptadores de rede e custos de coordenação entre processos passam a limitar a escalabilidade efetiva da aplicação, mesmo quando há paralelismo computacional disponível.

Para enfrentar esse desafio, o padrão *Message Passing Interface* (MPI), tradicionalmente empregado na coordenação entre processos em aplicações HPC, incorporou, a partir do MPI-2, a especificação MPI-IO, que provê primitivas para acesso paralelo a arquivos compartilhados [6]. Combinada a sistemas de arquivos paralelos como o Lustre, a MPI-IO permite que múltiplos processos gravem simultaneamente em regiões distintas de um mesmo arquivo, distribuindo a carga de E/S entre vários *Object Storage Targets* (OSTs) e contornando os gargalos típicos de arquiteturas com escritor centralizado [7].

Esse cenário é especialmente relevante para aplicações que combinam alto poder de computação paralela com produção intensiva de resultados, como ocorre nas ferramentas de planejamento da operação de sistemas elétricos. A crescente inserção de fontes renováveis intermitentes — solar e eólica — tem aumentado a complexidade dos modelos de despacho hidrotérmico e ampliado os volumes de dados produzidos, exigindo dessas aplicações tanto desempenho computacional quanto soluções eficazes de E/S [8, 9].

1.2 Motivação

O algoritmo *Stochastic Dual Dynamic Programming* (SDDP) [10] encontra amplo uso no planejamento e no despacho hidrotérmico em sistemas elétricos. No Brasil, o Operador Nacional do Sistema Elétrico (ONS) opera o Sistema Interligado Nacional (SIN) e utiliza o SDDP no contexto do Programa Mensal da Operação (PMO) para projetar a operação eletroenergética dos meses seguintes. O algoritmo decompõe o problema em uma malha bidimensional de subproblemas indexados por estágio e cenário; a fase de simulação final — etapa em que o algoritmo avalia a política já convergida em larga escala sobre um conjunto amplo de cenários — é naturalmente paralela, pois os cenários são mutuamente independentes e a aplicação pode distribuí-los entre processos MPI segundo o padrão *bag-of-tasks* [11].

A transição da operação programada para resolução horária, motivada pela necessidade de capturar adequadamente horários de ponta, vales e rampas associados à variabilidade das fontes intermitentes, multiplica em uma ordem de grandeza o volume de dados produzido por cenário e estágio. Nesse regime, a computação deixa de dominar exclusivamente o tempo de execução do SDDP, que passa a sofrer forte influência das operações de E/S associadas à persistência dos resultados. A arquitetura atual do SDDP concentra todas essas operações em um único processo escritor: cada cenário resolvido envia seu bloco de resultados, via `MPI_Send`, ao processo escritor, que serializa as gravações em disco. Essa estratégia introduz dois gargalos potenciais. O primeiro é o adaptador de rede do nó escritor, que recebe todas as transferências concorrentes dos processos trabalhadores. O segundo é o próprio caminho de escrita, serializado em um único processo, que não aproveita o paralelismo do sistema de arquivos subjacente.

A expansão dos ambientes computacionais utilizados no planejamento energético acentua esses gargalos. Além de supercomputadores tradicionais, como o Santos Dumont, do Laboratório Nacional de Computação Científica (LNCC), surge a alternativa da computação em nuvem, em particular a Amazon Web Services (AWS), que oferece serviços como o FSx for Lustre dedicados a cargas HPC. A portabilidade do SDDP entre esses ambientes torna ainda mais relevante o entendimento e a mitigação dos gargalos de E/S, já que cada plataforma impõe características distintas de hardware, rede e sistema de arquivos. Permanece em aberto, contudo, em que medida uma arquitetura de E/S baseada em escritas paralelas com MPI-IO sobre Lustre pode substituir o modelo com escritor único, sob quais condições os ganhos se materializam e como esses ganhos dependem da infraestrutura de armazenamento subjacente.

1.3 Objetivos e Contribuições

Esta dissertação propõe e avalia uma arquitetura paralela de E/S para o SDDP, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre, com o objetivo de substituir a estratégia atual com processo escritor único e investigar os ganhos de desempenho e escalabilidade obtidos.

Escopo. O foco deste trabalho é a *fase de simulação final* do SDDP — etapa executada após a convergência da política de operação, em que o algoritmo avalia um conjunto fixo de cortes de Benders em larga escala sobre múltiplos cenários para produzir os resultados operacionais entregues ao planejamento. Mais especificamente, o estudo concentra-se na *escrita dos resultados* dessa fase, em que se acumula o maior volume de dados produzido pela aplicação. Estão deliberadamente fora do escopo:

- a fase iterativa de geração de cortes (construção da política), incluindo suas escolhas algorítmicas;
- a paralelização das operações de *leitura* dos dados de entrada, cujo volume é muito inferior ao da escrita dos resultados horários e não constitui o gargalo investigado;
- eventuais melhorias no *escalonamento* de tarefas entre os *ranks* MPI, mantido conforme a estratégia *bag-of-tasks* já existente no SDDP.

Essa delimitação permite isolar o impacto da paralelização das escritas como variável de estudo, sem confundir os ganhos observados com efeitos de outras frentes de otimização.

As principais contribuições desta dissertação são:

- Diagnóstico dos gargalos da arquitetura atual de E/S do SDDP, com identificação do papel do processo escritor único e dos limites impostos pelo adaptador de rede do nó correspondente;
- Proposta de uma arquitetura de E/S paralela baseada em MPI-IO sobre Lustre, com escritas independentes por processo em *offsets* previamente calculados, preservando o formato dos arquivos de resultado consumidos pela cadeia de *softwares* do planejamento da operação;
- Instrumentação detalhada das duas implementações com métricas operacionalmente equivalentes, permitindo comparação direta entre tempo de computação, tempo de coordenação MPI, tempo de E/S, tempo de bloqueio e banda agregada percebida pela aplicação;

- Avaliação experimental comparativa em dois ambientes distintos — AWS com FSx for Lustre dedicado e Santos Dumont com Lustre compartilhado —, em configurações de 2 a 32 nós computacionais, identificando o regime em que a arquitetura proposta supera a atual e a magnitude dos ganhos obtidos;
- Análise da influência do número de *Object Storage Targets* (OSTs) na escalabilidade da E/S paralela, fornecendo orientações para o dimensionamento do sistema de arquivos em *deployments* futuros.

1.4 Organização

O Capítulo 2 introduz os principais conceitos necessários à compreensão desta dissertação: o padrão MPI e seus protocolos de comunicação, as operações paralelas de arquivos via MPI-IO, o sistema de arquivos paralelo Lustre, os ambientes computacionais utilizados nos experimentos — a nuvem da Amazon Web Services (AWS) e o supercomputador Santos Dumont (LNCC), com suas respectivas tecnologias de rede —, o Programa Mensal da Operação e o algoritmo SDDP, incluindo sua arquitetura atual de E/S. O Capítulo 3 detalha a proposta de paralelização dos mecanismos de E/S do SDDP, com o modelo de execução, a implementação das escritas independentes, as considerações de configuração do Lustre e a instrumentação adotada. O Capítulo 4 apresenta a descrição dos experimentos realizados nos ambientes AWS e Santos Dumont e a discussão dos resultados. O Capítulo 5 discute os trabalhos relacionados ao conteúdo desta dissertação. Finalmente, o Capítulo 6 resume os principais resultados e apresenta as conclusões, com as propostas de trabalhos futuros.

Capítulo 2

Fundamentação Teórica

2.1 MPI, protocolos de comunicação e operações paralelas de arquivos

O *Message Passing Interface* (MPI) é o padrão dominante para programação paralela baseada em processos que se comunicam por troca de mensagens. Em aplicações científicas executadas em *clusters* e supercomputadores, o MPI fornece tanto primitivas de comunicação ponto a ponto, como `MPI_Send` e `MPI_Recv`, quanto operações coletivas, como reduções, barreiras e difusões. A partir do MPI-2, o padrão também passou a incluir a especificação MPI-IO, que estende o mesmo modelo de coordenação entre processos para operações paralelas de entrada e saída em arquivos [1, 12].

Essa integração é importante porque comunicação e E/S não são problemas independentes em aplicações HPC. Uma arquitetura que concentra resultados em um único processo escritor transforma a E/S em comunicação ponto a ponto: os processos produtores enviam seus dados para um processo central, e esse processo serializa a escrita no sistema de arquivos. Por outro lado, uma arquitetura baseada em MPI-IO permite que os próprios processos produtores escrevam seus resultados, deslocando o gargalo do processo escritor para o sistema de arquivos paralelo. A escolha entre essas estratégias depende do tamanho das mensagens, do padrão de acesso aos dados, da regularidade dos deslocamentos no arquivo e da capacidade do sistema de armazenamento de atender requisições concorrentes.

No contexto da comunicação ponto a ponto, o comportamento de uma chamada de envio não é determinado apenas pela semântica da função usada pela aplicação, mas também pelo protocolo escolhido internamente pela biblioteca. Dois protocolos são particularmente relevantes: *eager* e *rendezvous*. No protocolo *eager*, a implementação envia pequenos blocos de dados de forma imediata, normalmente copiando-os para *buffers* internos até que o receptor execute a chamada correspondente. Esse caminho favorece baixa latência para mensagens pequenas, pois evita uma etapa

prévia de negociação, mas consome memória proporcional ao número de mensagens em trânsito.

No protocolo *rendezvous*, usado tipicamente para blocos maiores, o emissor primeiro negocia com o receptor antes da transferência efetiva dos dados. Esse *handshake* aumenta a latência inicial, porém reduz a necessidade de *buffers* intermediários e torna o consumo de memória mais previsível em cargas intensivas [13, 14]. Assim, mensagens pequenas e frequentes tendem a se beneficiar do *eager*, enquanto mensagens grandes favorecem o *rendezvous*, especialmente quando a aplicação opera próxima aos limites de memória ou de rede.

A Figura 2.1 ilustra esquematicamente as duas estratégias. No protocolo *eager*, o emissor transmite os dados sem negociação prévia, contando com *buffers* no lado do receptor para absorver eventuais descompassos entre a chegada da mensagem e a chamada de recepção. No protocolo *rendezvous*, o emissor envia primeiro um cabeçalho de controle e aguarda a confirmação do receptor (*Ready to Receive*) antes de iniciar a transferência efetiva dos dados, dispensando o uso de *buffers* intermediários no caminho de recepção.

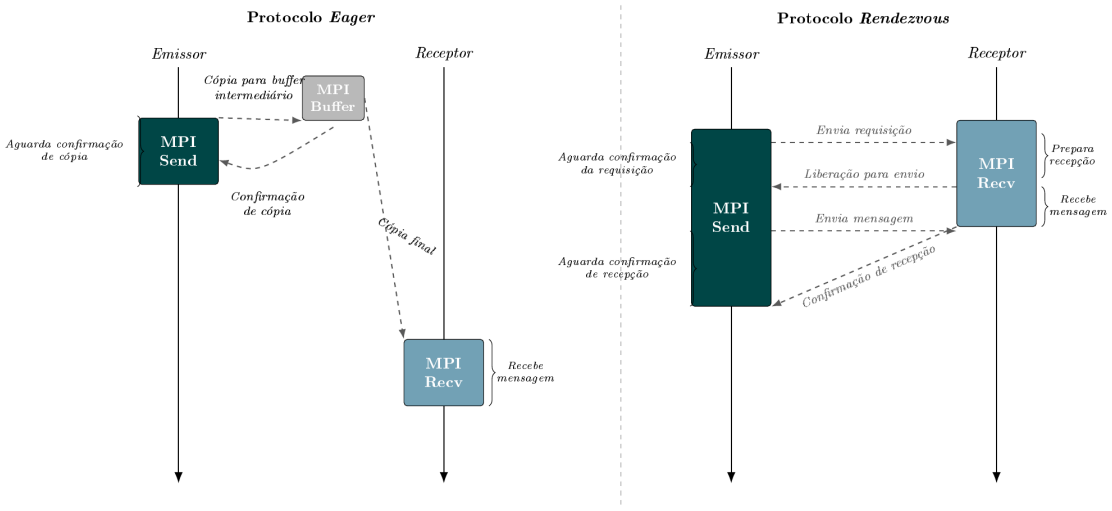


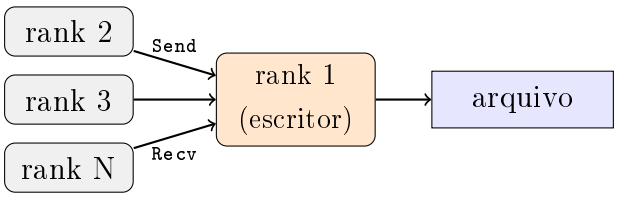
Figura 2.1: Comparação esquemática entre os protocolos *Eager* e *Rendezvous* em comunicação MPI ponto a ponto. No *Eager*, o emissor transmite os dados imediatamente, sem negociação prévia; no *Rendezvous*, há uma etapa de *handshake* entre emissor e receptor antes da transferência efetiva, eliminando a necessidade de *buffers* intermediários.

Um aspecto relevante em aplicações com padrão *bag-of-tasks* é o custo de bloqueio imposto ao remetente pela operação `MPI_Send`. Quando múltiplos *ranks* concorrem por um único receptor — como no modelo centralizado de E/S do SDDP —, o tempo de bloqueio cresce com o volume de dados enviado: o remetente fica

suspenso enquanto o receptor não disponibiliza capacidade de recepção. Esse comportamento se agrava à medida que o número de remetentes simultâneos aumenta, pois cada mensagem não processada introduz uma espera adicional para os demais trabalhadores.

A Figura 2.2 resume essa relação entre comunicação ponto a ponto e E/S paralela. O fluxo centralizado usa a rede para mover os resultados até um escritor único; o fluxo MPI-IO remove essa etapa intermediária e permite que os processos escrevam diretamente em regiões distintas do arquivo.

(a) Escritor centralizado



(b) Escrita paralela com MPI-IO

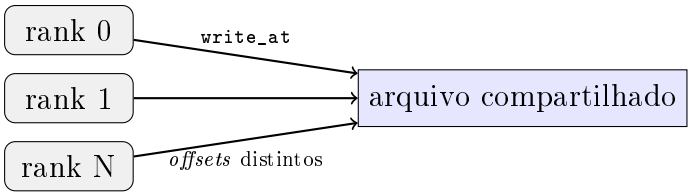


Figura 2.2: Relação entre comunicação ponto a ponto e E/S paralela. No modelo centralizado, os resultados passam por um processo escritor; no modelo MPI-IO, os próprios processos escrevem diretamente no arquivo.

Para operações paralelas de arquivos, o MPI-IO define uma API padronizada para que múltiplos processos acessem arquivos compartilhados de forma coordenada. As operações podem ser classificadas, de maneira prática, em três grupos: escritas independentes, escritas coletivas e escritas não bloqueantes ou assíncronas. Cada grupo tem custos e benefícios distintos, e a escolha correta depende do padrão de dados produzido pela aplicação.

Do ponto de vista do arquivo, dois padrões de acesso são particularmente relevantes: acesso contíguo e acesso com *strides*. No acesso contíguo, cada processo lê ou escreve uma região contínua do arquivo, como um bloco de resultados armazenado sequencialmente. Esse padrão é favorável ao sistema de arquivos paralelo, pois tende a gerar requisições maiores, alinhadas e com menor sobrecarga de metadados. No acesso com *strides*, por outro lado, os dados de um processo aparecem em regiões separadas por intervalos regulares, como ocorre em matrizes distribuídas por linhas, colunas ou blocos intercalados. Embora esse padrão represente de forma

natural muitas estruturas científicas, ele pode produzir várias requisições pequenas e espaçadas quando tratado de maneira ingênua. O MPI-IO permite descrever esses acessos não contíguos por meio de tipos derivados e *file views*, possibilitando que a biblioteca reorganize a E/S por técnicas como *data sieving* e *two-phase I/O* [1, 2, 15].

Nas escritas independentes, cada processo executa sua própria chamada de E/S sem exigir que os demais processos participem da mesma operação. Um exemplo típico é `MPI_File_write_at`, no qual o processo informa o *offset* do arquivo em que seus dados devem ser gravados. Esse padrão é adequado quando cada processo produz blocos naturalmente independentes, quando os tamanhos variam entre processos ou quando a aplicação não possui pontos convenientes de sincronização global. Também é uma escolha simples e robusta para aplicações do tipo *bag-of-tasks* [11], em que tarefas independentes terminam em momentos diferentes e podem persistir seus resultados sem aguardar as demais.

O custo das escritas independentes é que o sistema de arquivos pode receber muitas requisições pequenas, desalinhadas ou concorrentes. Em sistemas paralelos como o Lustre, isso pode funcionar bem quando os blocos são suficientemente grandes e os *offsets* estão bem distribuídos entre *stripes* e OSTs. Entretanto, quando muitos processos escrevem pequenas regiões dispersas, a sobrecarga de metadados e a contenção nos servidores de armazenamento podem limitar a escalabilidade.

Nas escritas coletivas, todos os processos de um comunicador participam da mesma operação, como em `MPI_File_write_all`. Nesse caso, a biblioteca MPI-IO tem uma visão global dos acessos e pode reorganizar a E/S por meio de técnicas como *two-phase I/O*, agregação de requisições e *data sieving*. Em vez de cada processo enviar pequenas escritas diretamente ao sistema de arquivos, alguns processos agregadores podem reunir dados de vários processos e emitir operações maiores, mais contíguas e melhor alinhadas ao particionamento do arquivo.

Esse padrão é recomendado quando a aplicação possui fases bem definidas de produção de dados, quando todos ou quase todos os processos escrevem em uma mesma etapa e quando os acessos formam uma estrutura regular no arquivo, como matrizes distribuídas, campos de simulação em grades, *checkpoints* globais e saídas por passo de tempo. A escrita coletiva tende a reduzir o número de chamadas ao sistema de arquivos e pode aumentar a banda efetiva, mas introduz sincronização entre processos. Se alguns deles produzem pouco dado, atrasam a chegada à chamada coletiva ou têm padrões de acesso muito irregulares, a operação coletiva pode aumentar o tempo de espera e prejudicar a sobreposição com a computação.

O MPI-IO também oferece variantes não bloqueantes, como `MPI_File_iwrite_at` e `MPI_File_iwrite_all`. Nessas operações, a chamada inicia a E/S e retorna antes de sua conclusão, permitindo que a aplicação execute computação ou comunicação enquanto a escrita progride. A aplicação verifica posteriormente a conclusão

por meio de chamadas como `MPI_Wait` ou `MPI_Test`. Esse modelo é útil quando há trabalho computacional independente logo após a geração dos dados, pois permite sobrepor parte do custo de E/S com computação. Contudo, operações assíncronas não eliminam o custo da escrita: elas apenas mudam o ponto em que a aplicação espera por sua conclusão. Além disso, a aplicação precisa preservar os *buffers* até o término da operação e controlar o número de escritas pendentes para evitar pressão excessiva de memória ou filas de E/S.

A Figura 2.3 contrasta os dois padrões principais de escrita. Na escrita independente, cada processo emite sua própria operação diretamente para o arquivo. Na escrita coletiva, a biblioteca MPI-IO coordena os processos e pode transformar várias requisições pequenas em operações agregadas.

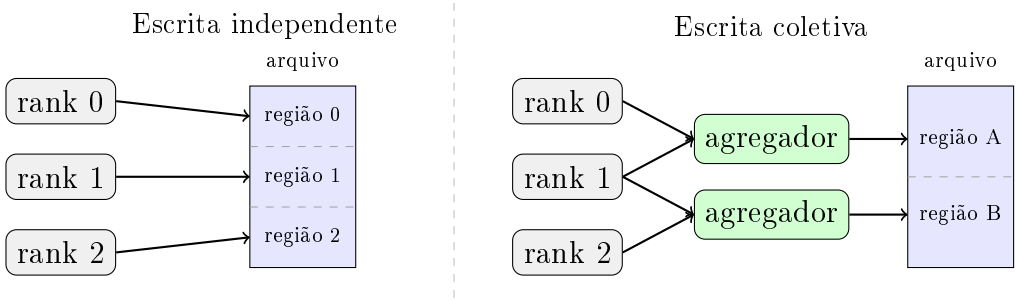


Figura 2.3: Diferença entre escritas independentes e coletivas em MPI-IO. Escritas independentes preservam autonomia dos processos; escritas coletivas permitem agregação e reordenação das requisições pela biblioteca MPI-IO.

Tabela 2.1: Critérios práticos para escolha do modo de escrita em MPI-IO.

Modo de escrita	**Padrão de aplicação mais adequado**
Independente	Tarefas independentes, tamanhos de dados variáveis, término assíncrono dos *ranks*, *offsets* conhecidos, blocos contíguos por processo e baixa necessidade de sincronização global.
Coletiva	Fases globais de saída, dados regulares ou quase regulares, muitos acessos pequenos ou com *strides* que podem ser agregados, *checkpoints* e matrizes ou campos distribuídos.
Assíncrona independente	Aplicações com trabalho local após a geração dos dados e escritas por processo que podem progredir em segundo plano sem exigir participação imediata dos demais processos.
Assíncrona coletiva	Aplicações com fases coletivas bem definidas e computação útil após o início da E/S, desde que os *buffers* possam permanecer válidos até a conclusão da operação.

2.2 Sistema de arquivos paralelo Lustre

O Lustre é um sistema de arquivos paralelo projetado especificamente para cargas massivas de HPC, no qual a separação entre o fluxo de metadados e o fluxo de dados é o princípio central de sua arquitetura. Essa separação permite que múltiplos clientes acessem o armazenamento de forma paralela e balanceada, escalando a banda agregada com o número de servidores disponíveis [7, 16]. A Figura 2.4 apresenta uma visão geral dos componentes do sistema, descritos a seguir.

O *Metadata Server* (MDS) é o componente responsável pelos metadados do sistema de arquivos, isto é, pelo *namespace* (estrutura de diretórios, nomes de arquivos), pelas permissões e pela informação sobre em quais alvos de armazenamento residem os dados de cada arquivo. O Lustre persiste os metadados propriamente ditos em um ou mais *Metadata Targets* (MDTs), os dispositivos de bloco associados ao MDS. Quando um cliente abre um arquivo, ele primeiro consulta o MDS para localizar os dados; uma vez obtida essa informação, as operações subsequentes de leitura e escrita ocorrem diretamente com os servidores de dados, sem passar novamente pelo MDS. Isso evita que o servidor de metadados se torne um gargalo durante as transferências de dados.

O Lustre realiza o armazenamento dos dados por meio dos *Object Storage Servers* (OSSs), que gerenciam o acesso aos *Object Storage Targets* (OSTs) — os dispositivos

de bloco onde o sistema grava efetivamente os dados dos arquivos. Cada OSS administra um ou mais OSTs, e o conjunto de OSTs define a capacidade total e a banda agregada de E/S do sistema. Como os clientes se comunicam simultaneamente com vários OSSs, o paralelismo de acesso cresce com o número de OSTs, sendo este um dos principais fatores de escalabilidade do Lustre.

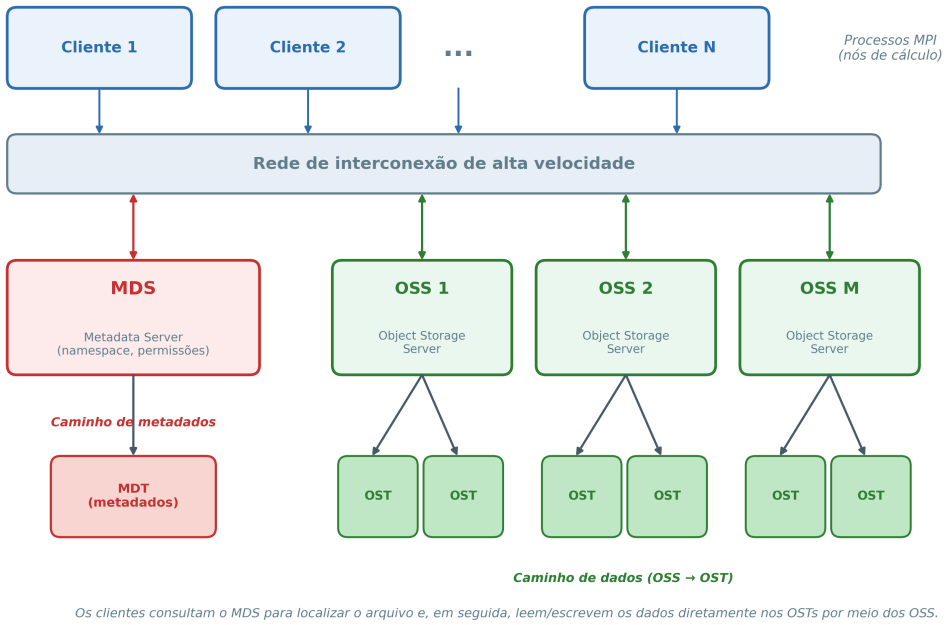


Figura 2.4: Arquitetura do sistema de arquivos paralelo Lustre. O servidor de metadados (MDS/MDT) mantém o *namespace*, enquanto o Lustre distribui os dados entre os OSTs gerenciados pelos OSSs. Os clientes consultam o MDS para localizar o arquivo e, em seguida, acessam os dados diretamente nos OSTs.

Para distribuir os dados entre os OSTs, o Lustre emprega a técnica de *striping*, na qual divide um arquivo em blocos de tamanho fixo (*stripes*) e os grava de forma circular (*round-robin*) em um subconjunto de OSTs, conforme ilustrado na Figura 2.5. Dois parâmetros controlam esse comportamento: o *stripe size*, que define o tamanho de cada bloco, e o *stripe count*, que indica em quantos OSTs o Lustre espalha o arquivo. Ao distribuir um único arquivo por vários OSTs, o *striping* permite que a aplicação leia ou escreva diferentes partes em paralelo, multiplicando a banda efetiva percebida pela aplicação.

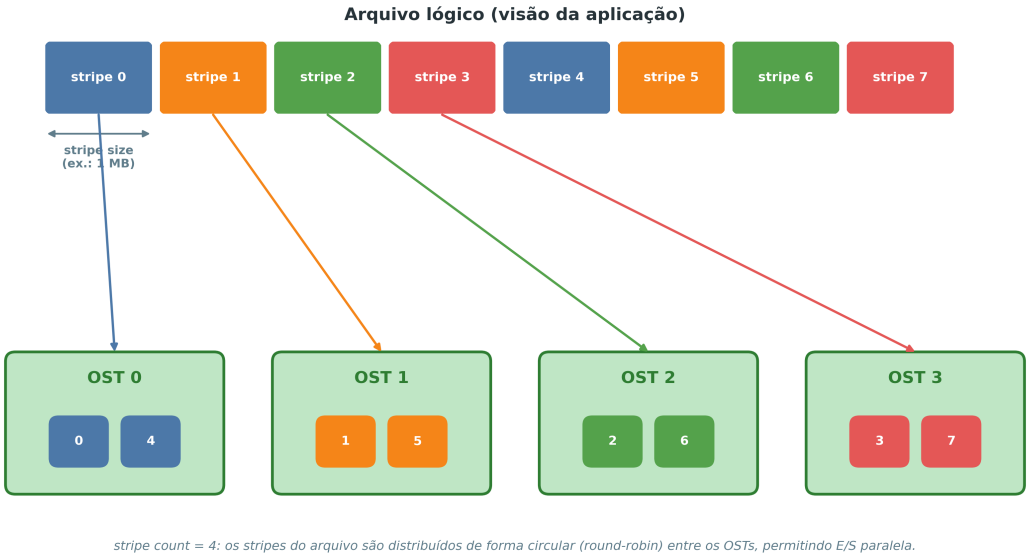


Figura 2.5: Distribuição de um arquivo em *stripes* pelos OSTs no Lustre. Com *stripe count* igual a 4, o Lustre aloca os blocos de forma circular entre os OSTs, viabilizando o acesso paralelo aos dados.

A escolha desses parâmetros deve considerar o padrão de acesso da aplicação, pois influencia diretamente o *throughput* e a escalabilidade: *stripes* maiores e *stripe counts* menores tendem a favorecer grandes operações sequenciais, enquanto *stripes* menores distribuídos por mais OSTs podem beneficiar acessos concorrentes e de granularidade fina, embora aumentem a sobrecarga de coordenação entre os alvos [16].

2.3 Ambientes computacionais

Este trabalho conduziu os experimentos em dois ambientes de computação de alto desempenho (HPC) com características distintas: a nuvem pública da Amazon Web Services (AWS) e o supercomputador Santos Dumont (SDumont). Embora ambos ofereçam recursos comparáveis de processamento e armazenamento paralelo, eles se diferenciam de forma marcante na tecnologia de interconexão entre nós, fator determinante para o desempenho de aplicações paralelas. A Ethernet, tradicionalmente projetada para redes locais corporativas, evoluiu para padrões de alta velocidade e oferece elevada taxa de transferência (*bandwidth*), porém apresenta latências relativamente maiores, frequentemente na ordem de dezenas de microssegundos. Já o InfiniBand foi desenvolvido especificamente para ambientes de HPC, combinando altas larguras de banda com latências extremamente baixas, tipicamente abaixo de 2 μs, graças a recursos como comunicação RDMA (*Remote Direct Memory Access*).

Essa diferença torna o InfiniBand preferido em supercomputadores e *clusters* científicos, enquanto a Ethernet, pelo menor custo e ampla compatibilidade, predomina em *datacenters* e ambientes de nuvem [17, 18]. As subseções a seguir descrevem cada ambiente e suas respectivas tecnologias de rede.

2.3.1 Ambiente AWS

A Amazon Web Services (AWS) tem se consolidado como uma das principais plataformas de computação em nuvem para aplicações de *High Performance Computing* (HPC), oferecendo recursos sob demanda que permitem a execução de cargas de trabalho intensivas em computação e E/S. Para atender aplicações intensivas em dados, a AWS integra sistemas de armazenamento de alto desempenho, como o Amazon FSx for Lustre, que viabiliza acesso paralelo e em larga escala, semelhante a sistemas de arquivos utilizados em supercomputadores tradicionais [19, 20].

No que se refere à rede, as instâncias da AWS empregam o *Elastic Network Adapter* (ENA), uma interface de rede virtual de alto desempenho baseada em Ethernet, que oferece largura de banda elevada e baixa sobrecarga de processador por meio de recursos como *checksum offload* e distribuição de tráfego em múltiplas filas. O ENA constitui a base da conectividade entre instâncias e do acesso ao FSx for Lustre, sustentando a comunicação MPI utilizada nos experimentos [21].

Por se tratar de uma infraestrutura virtualizada e compartilhada, a AWS não expõe diretamente ao usuário a topologia física dos nós. Ela organiza seus recursos em *regiões*, subdivididas em *zonas de disponibilidade* (*Availability Zones*) — conjuntos isolados de *datacenters* com energia, refrigeração e rede independentes. Instâncias em uma mesma zona de disponibilidade apresentam latência de rede mais baixa entre si, razão pela qual este trabalho alocou todos os nós dos experimentos em uma única zona. Para reduzir ainda mais a latência de comunicação, este trabalho utilizou um *Placement Group* do tipo *cluster*, mecanismo que agrupa as instâncias logicamente próximas dentro da mesma zona de disponibilidade, em um segmento de rede de baixa latência e alta vazão. Essa configuração aproxima o desempenho de comunicação da nuvem ao de um *cluster* dedicado, e a AWS a recomenda para cargas de trabalho de HPC fortemente acopladas, como as do SDDP [22]. Este trabalho também provisionou o próprio sistema de arquivos Amazon FSx for Lustre em uma única zona de disponibilidade, coincidente com a das instâncias de cálculo, de modo que o acesso ao armazenamento permaneça no mesmo segmento de rede de baixa latência, evitando o tráfego entre zonas distintas.

2.3.2 Ambiente de supercomputação: Santos Dumont

O supercomputador Santos Dumont (SDumont), instalado no Laboratório Nacional de Computação Científica (LNCC), em Petrópolis-RJ, atua como nó central (Tier-0) do Sistema Nacional de Processamento de Alto Desempenho (SINAPAD) e representa um ambiente de HPC tradicional, com infraestrutura dedicada e exclusiva para aplicações científicas. Diferentemente da nuvem, sua topologia é fixa e conhecida, e uma rede de interconexão de alta velocidade baseada em InfiniBand interliga seus nós.

O InfiniBand é a tecnologia de interconexão predominante em supercomputadores, oferecendo largura de banda elevada e, sobretudo, latência muito baixa na troca de mensagens entre processos. Tais características são particularmente relevantes para aplicações com troca intensa de mensagens e operações coletivas de E/S, como o SDDP, em que a baixa latência na comunicação entre *ranks* MPI é tão importante quanto a banda disponível [17, 18].

Embora o InfiniBand suporte nativamente a comunicação RDMA (*Remote Direct Memory Access*) — que permite o acesso direto à memória de nós remotos sem intervenção do sistema operacional — este trabalho não explorou esse caminho em seus experimentos. Utilizou a biblioteca MPICH versão 3.2 com o dispositivo de comunicação *ch3*, no qual a troca de mensagens ocorre sobre a pilha de rede convencional (IP sobre InfiniBand) e não pela interface nativa de *verbs*/RDMA. Dessa forma, o ambiente SDumont se beneficia da elevada banda e da baixa latência do InfiniBand, mas sem o ganho adicional de cópia zero proporcionado pelo RDMA.

2.4 O Programa Mensal da Operação (PMO)

No setor elétrico brasileiro, o Programa Mensal da Operação (PMO) é um documento técnico elaborado pelo Operador Nacional do Sistema Elétrico (ONS) que estabelece as diretrizes de curto prazo para a operação do Sistema Interligado Nacional (SIN). O PMO contempla projeções de carga, disponibilidade de geração, restrições de transmissão e condições hidrológicas, servindo como referência para a programação da operação hidrotérmica de forma a garantir o equilíbrio entre oferta e demanda de energia elétrica. Além disso, define metas de despacho de usinas hidrelétricas e termelétricas, contribuindo para a segurança energética e para a otimização do uso dos recursos disponíveis. Por sua relevância, o PMO orienta não apenas as decisões operativas do ONS, mas também agentes do setor, incluindo geradores, comercializadores e distribuidores, sendo um instrumento fundamental para a gestão integrada e eficiente do SIN [23].

2.5 Algoritmo SDDP

O algoritmo SDDP (*Stochastic Dual Dynamic Programming*), proposto por [10], decompõe o problema de planejamento hidrotérmico em uma malha bidimensional de subproblemas determinísticos: o algoritmo indexa cada subproblema por um par (estágio, cenário), em que os estágios representam períodos discretos da operação — meses, no caso da operação programada do SIN — e os cenários representam realizações estocásticas das variáveis relevantes, principalmente afluências aos reservatórios. O algoritmo constrói a solução do problema em duas fases. Na fase de *convergência*, iterações *forward*/*backward* constroem progressivamente uma política de operação representada por um conjunto de cortes de Benders, até atingir um critério de convergência. Na fase de *simulação final*, executada após a convergência, o algoritmo avalia a política já fixa em larga escala sobre um conjunto amplo de cenários para produzir os resultados operacionais entregues ao planejamento — variáveis primais, multiplicadores duais, custos e despachos. Esta dissertação concentra-se exclusivamente na fase de simulação final.

Nessa fase, os cenários são, por construção, mutuamente independentes: cada cenário corresponde a uma realização estocástica distinta, avaliada sob a mesma política convergida, sem compartilhamento de estado nem coordenação durante a execução. Essa propriedade caracteriza a fase de simulação final do SDDP como uma aplicação do tipo *bag-of-tasks* [11]: um conjunto de tarefas mutuamente independentes que podem ser distribuídas entre processadores sem sincronização durante a execução. A aplicação particiona os cenários entre os *ranks* MPI e os resolve em paralelo, de modo que o tempo total da fase de simulação fica limitado pelo cenário mais lento e não pela soma sequencial de todos. A natureza *bag-of-tasks* da fase de simulação é o que torna esta etapa do SDDP escalável em sistemas HPC; é também o que desloca o gargalo da computação para a persistência dos resultados, à medida que o número de cenários cresce.

Após a resolução, a aplicação precisa persistir em disco os resultados de cada subproblema — variáveis primais, multiplicadores duais e custos —, pois eles alimentam etapas subsequentes do próprio algoritmo e análises estatísticas posteriores.

A granularidade temporal adotada para os resultados de cada estágio tem impacto direto sobre o volume de dados produzido por iteração. A abordagem tradicional do SDDP agrega a operação dentro de cada estágio em poucos blocos representativos de carga (*patamares*: pesado, médio e leve), reduzindo cada estágio a um pequeno número de pontos operativos por cenário. A operação programada moderna, entretanto, especialmente com a crescente penetração de fontes intermitentes como solar e eólica, requer resolução horária para capturar adequadamente horários de ponta, vales e rampas associadas à variabilidade dessas fontes. Esta

dissertação foca exatamente no regime de resultados horários, em que cada estágio de um cenário gera dados proporcionais ao número de horas simuladas — ordem de magnitude maior que o caso tradicional por blocos. É nesse regime que o gargalo de E/S se torna efetivamente dominante: o volume agregado escrito por iteração cresce linearmente com o número de cenários, com o número de estágios e com a granularidade temporal interna de cada estágio, e a estratégia de implementação dessas escritas determina diretamente a escalabilidade da aplicação em ambientes HPC.

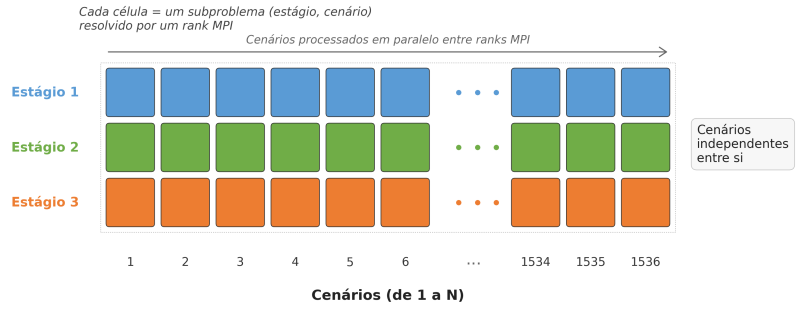


Figura 2.6: Estrutura de cenários e estágios do SDDP. Cada célula corresponde a um subproblema $(\text{estágio}, \text{cenário})$. Os cenários são mutuamente independentes e podem ser distribuídos entre *ranks* MPI, caracterizando uma aplicação *bag-of-tasks*.

2.5.1 Comparação dos formatos dos resultados

O SDDP organiza seus resultados em arquivos associados aos estágios, cenários e elementos do sistema elétrico. Cada elemento corresponde a uma entidade física ou lógica modelada na base de dados do sistema elétrico, como usinas hidrelétricas, usinas térmicas, reservatórios, barras, interligações ou outros componentes cujas grandezas a simulação registra em sua saída. Assim, a estrutura dos arquivos depende tanto da granularidade temporal adotada quanto da quantidade de elementos representados no sistema analisado.

No formato por patamares, o SDDP agrega os resultados de cada par $(\text{estágio}, \text{cenário})$ em poucos blocos representativos de carga, como pesado, médio e leve. Esse formato reduz o número de registros por estágio e cenário, mas também limita a resolução temporal disponível para análises posteriores. No formato horário, por outro lado, o SDDP detalha cada estágio em horas individuais. Para um estágio mensal de 30 dias, por exemplo, cada par $(\text{estágio}, \text{cenário})$ pode gerar 720 registros, um para cada hora, aumentando significativamente o volume de dados produzido.

Em ambos os formatos, os registros mantêm colunas de identificação seguidas pelos valores dos elementos do sistema elétrico. A diferença principal está na dimen-

são temporal interna ao estágio: no formato por patamares, um pequeno conjunto de blocos de carga representa essa dimensão; no formato horário, uma sequência de horas a representa. A Figura 2.7 compara os dois formatos de maneira esquemática.

Formato por patamares

estágio	cenário	patamar	E_1	$\cdots$	E_N
e_1	c_1	pesado	v_1	$\cdots$	v_N
e_1	c_1	médio	v_1	$\cdots$	v_N
e_1	c_1	leve	v_1	$\cdots$	v_N

Formato horário

estágio	cenário	hora	E_1	$\cdots$	E_N
e_1	c_1	1	v_1	$\cdots$	v_N
e_1	c_1	2	v_1	$\cdots$	v_N
$\vdots$	$\vdots$	$\cdots$	$\vdots$	$\ddots$	$\vdots$
e_1	c_1	720	v_1	$\cdots$	v_N

Figura 2.7: Comparação esquemática entre os formatos de resultados por patamares e horário do SDDP.

2.5.2 Arquitetura atual de E/S do SDDP

Na arquitetura atual do SDDP, apenas um *rank* MPI (o *rank* 1, denominado processo escritor) realiza todas as operações de E/S do sistema. O *rank* 0 atua como processo distribuidor, coordenando o envio de tarefas e configurações aos demais *ranks*. Cada nó de computação, após resolver sua respectiva tarefa, envia os resultados por meio de um *buffer* de dados utilizando a função `MPI_Send`. O processo escritor então recebe esse *buffer* por meio de uma chamada `MPI_Recv` e, posteriormente, grava os dados em um sistema de arquivos distribuído. A Figura 2.8 ilustra o funcionamento dessa arquitetura.

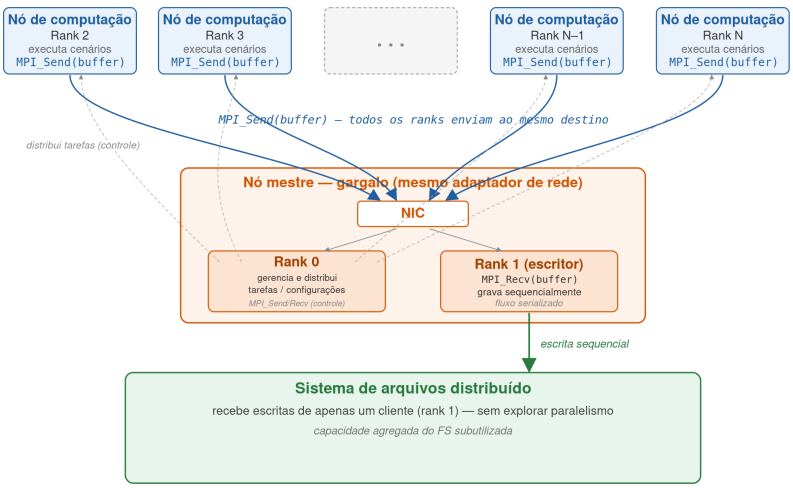


Figura 2.8: Arquitetura atual de E/S do SDDP com o *rank* 0 atuando como processo distribuidor e o *rank* 1 como processo escritor.

Independentemente do número de nós computacionais utilizados, apenas um

processo é responsável pelas operações de E/S. Cada nó utiliza sua banda efetiva para enviar os *buffers* de dados, enquanto o processo escritor utiliza sua banda para recebê-los. A primeira hipótese para as limitações de escalabilidade nessa arquitetura está relacionada ao adaptador de rede do servidor que executa o processo escritor.

Adicionalmente, o *rank* 0 é responsável pelo gerenciamento e controle da aplicação, como a distribuição de tarefas para os demais *ranks* e o envio/recebimento das configurações do sistema. Essas atividades também utilizam o mesmo adaptador de rede empregado na recepção dos *buffers* de dados. Outra hipótese é que o processo escritor opera continuamente em um fluxo sequencial de recepção e escrita dos dados, o que pode representar um gargalo adicional, sobretudo em sistemas com alto volume de dados. Por fim, um atraso do processo escritor na recepção ou na gravação dos *buffers* de dados pode obrigar os nós computacionais a aguardar, gerando sobrecarga no envio dos *buffers* e aumentando o custo computacional da execução paralela.

Esta dissertação denomina o *rank* 0 processo distribuidor, o *rank* 1 processo escritor, e os demais *ranks*, responsáveis pela resolução dos cenários, processos trabalhadores.

Capítulo 3

Paralelização dos Mecanismos de E/S do SDDP

Este capítulo apresenta a proposta de paralelização dos mecanismos de entrada e saída (E/S) do SDDP.

3.1 Visão geral da arquitetura

A solução substitui o modelo atual por uma organização baseada em MPI-IO sobre o sistema de arquivos Lustre. Nessa abordagem, cada processo grava diretamente seus próprios resultados em regiões independentes do espaço de saída, evitando a concentração das operações de E/S em um único processo.

O ponto central da proposta é explorar, simultaneamente, o paralelismo da aplicação MPI e o paralelismo do sistema de arquivos. Como os resultados produzidos por diferentes processos são independentes, a aplicação pode posicionar suas escritas previamente e executá-las em paralelo. O Lustre distribui os arquivos em *stripes* entre diferentes *Object Storage Targets* (OSTs); assim, o sistema de arquivos pode atender múltiplas escritas independentes em OSTs distintos, aumentando a largura de banda agregada e reduzindo a contenção observada no modelo com escritor centralizado.

Dessa forma, a proposta visa reduzir dois gargalos da arquitetura original. O primeiro é o gargalo de comunicação, pois a aplicação deixa de enviar os dados ao processo escritor antes de persisti-los. O segundo é o gargalo de armazenamento, pois a proposta passa a distribuir a escrita entre os recursos do sistema de arquivos paralelo, em vez de serializá-la em um único processo.

A Figura 3.1 apresenta uma visão geral da arquitetura proposta. O *rank* 0 atua como processo distribuidor, responsável por coordenar a distribuição das tarefas, enquanto os demais processos gravam seus resultados de forma independente sobre

arquivos compartilhados. Nessa abstração, múltiplos processos operam concorrentemente sobre o mesmo arquivo lógico em *offsets* distintos por meio das primitivas do MPI-IO [12].

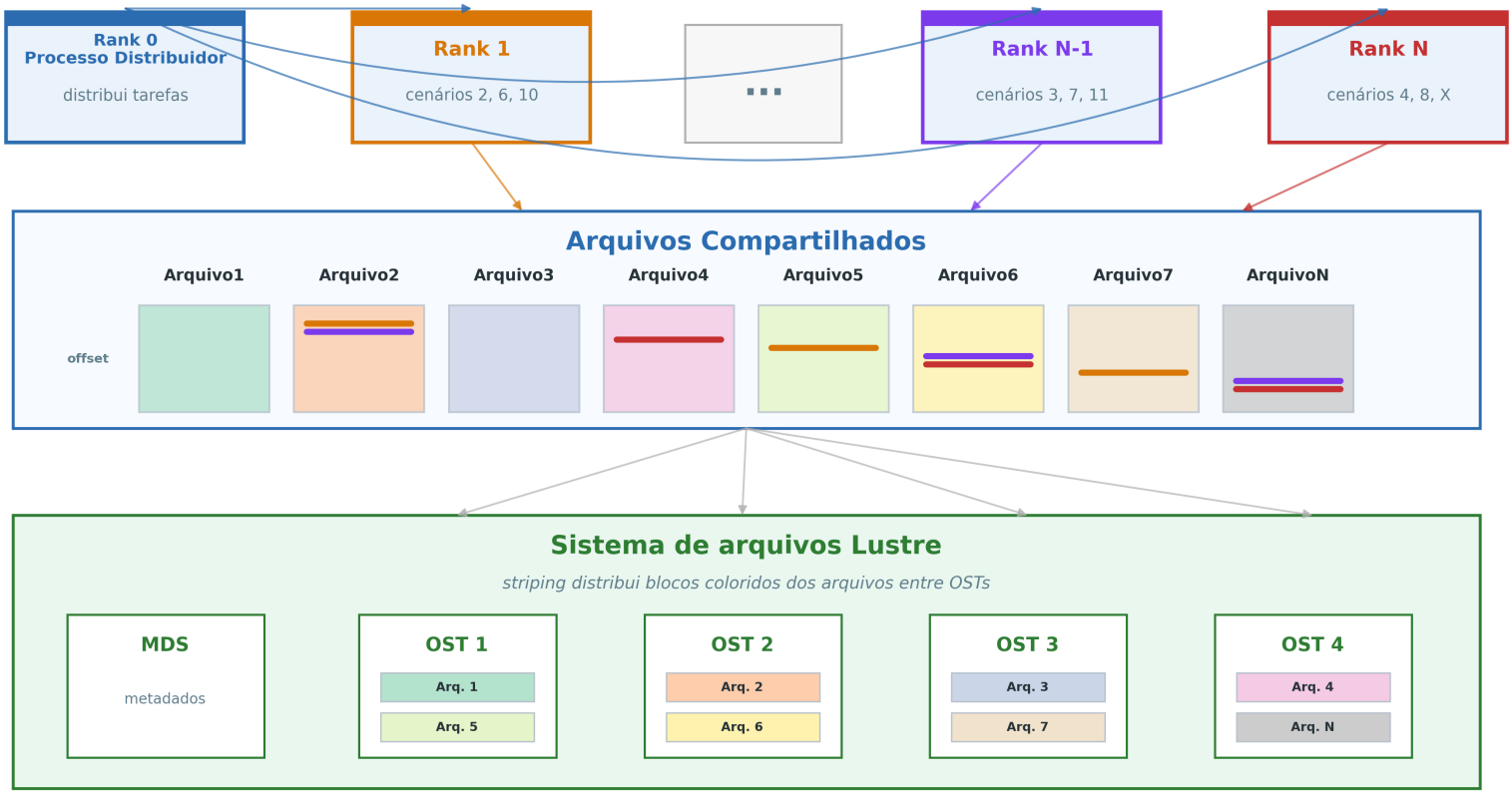


Figura 3.1: Arquitetura proposta com MPI-IO e Lustre, com o *rank* 0 atuando como processo distribuidor e os demais processos escrevendo diretamente em arquivos compartilhados.

3.2 Modelo de execução

A Figura 3.2 ilustra a linha do tempo da execução do SDDP segundo a arquitetura proposta. Essa representação destaca a separação entre o processo distribuidor, a chamada coletiva `MPI_File_open` entre os *ranks* trabalhadores, as escritas independentes e a chamada `MPI_File_close`. Com isso, a figura evidencia que a coordenação global fica restrita aos pontos de abertura e fechamento dos arquivos, enquanto as escritas realizadas após a abertura permanecem independentes.

O processo distribuidor representa o *rank* 0, responsável por distribuir as tarefas aos demais *ranks*. Quando não está enviando uma nova tarefa, esse *rank* permanece aguardando o recebimento das tarefas concluídas pelos *ranks* trabalhadores. Por esse motivo, a figura representa sua linha integralmente como comunicação.

Antes da primeira escrita, os *ranks* trabalhadores participam de uma operação

coletiva de abertura dos arquivos de resultado. Esse ponto de chamada `MPI_File_open` atua como uma barreira entre os trabalhadores: processos que chegam antes aguardam os demais antes de iniciar as escritas independentes. O processo distribuidor não participa dessa chamada coletiva.

Cada *rank* trabalhador executa o ciclo *bag-of-tasks* [11]: recebe do processo distribuidor uma tarefa correspondente a um cenário, resolve a simulação desse cenário e, ao término, efetua escritas independentes nos diversos arquivos de resultado, persistindo os blocos no formato proprietário do modelo SDDP. O tamanho de cada bloco varia conforme o arquivo de resultado produzido. Concluída a persistência, o processo solicita uma nova tarefa ao distribuidor, repetindo esse ciclo enquanto houver cenários disponíveis.

Como o tempo de resolução de cada cenário pode variar, a distribuição dinâmica de tarefas conduz a um desbalanceamento natural da carga de escrita entre os *ranks* trabalhadores: alguns processos concluem — e persistem — mais cenários que outros ao longo da execução. Na Figura 3.2, esse efeito aparece no *rank* 2, que executa mais ciclos de simulação e escrita do que seus pares no mesmo intervalo. Esse desbalanceamento, intrínseco ao padrão *bag-of-tasks*, dificulta a adoção de escritas coletivas [24]: chamadas coletivas exigem que todos os *ranks* participem conjuntamente da mesma operação de E/S, o que forçaria os processos mais rápidos a aguardar a chegada dos mais lentos a cada ponto coletivo, anulando parte do paralelismo ganho com a distribuição dinâmica. Em casos extremos, esse desencontro pode inclusive provocar *deadlock*: se algum *rank* já houver esgotado suas tarefas e abandonado o ciclo enquanto outros ainda possuem cenários a persistir, a chamada coletiva subsequente ficaria bloqueada indefinidamente, à espera de um participante que jamais chegaria. Por essas razões, a estratégia de escritas independentes mostra-se mais aderente ao modelo de execução adotado.

Quando não há mais cenários a resolver, a aplicação inicia uma nova chamada `MPI_File_close`, responsável pelo fechamento coletivo dos arquivos de resultado.



Figura 3.2: Linha de tempo da simulação final do SDDP com MPI-IO. O *rank* 0 atua apenas como processo distribuidor; os *ranks* trabalhadores executam cenários, sincronizam na chamada coletiva `MPI_File_open` antes da primeira escrita, realizam escritas independentes e sincronizam novamente em `MPI_File_close`.

3.3 Detalhes de implementação

Os módulos de acesso a dados do SDDP, responsáveis pelas rotinas de leitura e escrita dos arquivos de entrada e saída, constituem uma biblioteca `C++`. Foi nesse ponto da aplicação que este trabalho incorporou a solução baseada em MPI-IO, substituindo o fluxo centralizado de escrita por operações paralelas realizadas diretamente pelos processos MPI. Concentrar a mudança nesse módulo permitiu introduzir a nova estratégia de E/S sem alterar a estrutura principal do algoritmo: a proposta preservou a lógica de simulação e restringiu as modificações ao caminho de persistência dos resultados. Na prática, as rotinas MPI-IO abrem os arquivos de resultado de forma compartilhada e permitem que cada processo grave diretamente os blocos produzidos pela simulação em posições previamente calculadas, conforme a organização lógica descrita na Seção 3.3.2.

3.3.1 Implementação das escritas independentes

Como a simulação de cada cenário transcorre de forma independente, a abordagem baseada em MPI-IO permite que cada operação de escrita também seja inde-

pendente: cada processo grava em um *offset* específico, determinado pelo cenário resolvido, sem necessidade de sincronização explícita entre escritas concorrentes.

A implementação adota escritas independentes de MPI-IO, combinando chamadas assíncronas e síncronas conforme o tamanho do bloco de dados. Para blocos menores que 128 KB, a implementação utiliza operações assíncronas, permitindo que o processo solicite a escrita e retome a execução sem aguardar imediatamente a conclusão da operação. Para blocos iguais ou superiores a 128 KB, a implementação emprega operações síncronas, nas quais o processo aguarda a conclusão da escrita antes de prosseguir.

A escolha de combinar chamadas assíncronas e síncronas separadas por um limite de tamanho fundamenta-se na assimetria entre os custos típicos de chamadas assíncronas e síncronas no MPI-IO: para blocos pequenos, o custo fixo de coordenação tende a dominar o tempo efetivo de transferência, beneficiando o despacho não-bloqueante; para blocos significativamente maiores, o tempo de transferência supera o custo fixo, reduzindo o ganho da assincronia e aumentando a pressão sobre *buffers* internos e *request slots* pendentes [2, 13].

Este trabalho adota, portanto, um limite de 128 KB para separar os dois caminhos, seguindo uma racionalidade conceitualmente análoga à dos protocolos Eager e Rendezvous em comunicação MPI ponto-a-ponto [13, 14]: tratamento imediato e leve para mensagens pequenas, fluxo controlado para mensagens grandes. Trata-se de um parâmetro de projeto desta dissertação; o refinamento empírico desse valor (por exemplo, experimentos de sensibilidade variando o limite entre 64, 128 e 256 KB) fica como direção natural de continuidade.

A Figura 3.3 resume o fluxo de decisão adotado na implementação. Após o cálculo do *offset* correspondente ao cenário, o tamanho do bloco determina a rotina de escrita utilizada: blocos menores que 128 KB seguem pelo caminho assíncrono, enquanto blocos iguais ou superiores a esse limite seguem pelo caminho síncrono.

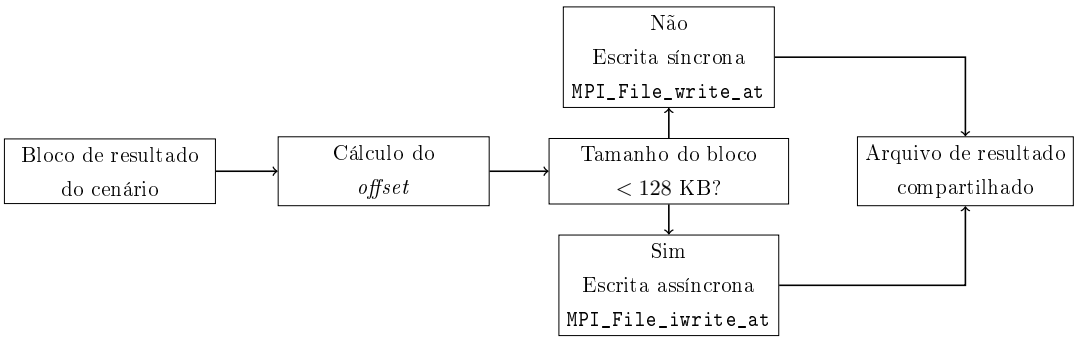


Figura 3.3: Fluxo de decisão para escritas independentes com MPI-IO (limite de 128 KB adotado neste trabalho como parâmetro de projeto, conforme discussão no texto).

Essa estratégia também facilita a comparação com a implementação atual. Como a arquitetura original já apresenta comportamento distinto para mensagens de tamanhos diferentes, separar os blocos pequenos e grandes na implementação MPI-IO permite avaliar a nova solução sob uma lógica compatível com o perfil de comunicação observado anteriormente, isolando melhor o impacto da mudança no mecanismo de escrita.

Como cada cenário ocupa um *offset* próprio em cada arquivo de resultado, esse arranjo elimina o risco de escritas concorrentes incidirem sobre uma mesma região. Essa propriedade constitui um benefício relevante da implementação proposta, pois dispensa o emprego de mecanismos de bloqueio (*locks*) para assegurar a corretude das operações concorrentes. Na implementação, o processo calcula o *offset* de cada escrita a partir da posição lógica do cenário no arquivo (conforme detalhado na Seção 3.3.2), do tamanho do registro associado à combinação (estágio, cenário, hora) e da quantidade de elementos do sistema elétrico armazenados em cada registro.

Para os arquivos no formato horário, nos quais cada registro corresponde a uma hora dentro de um par (estágio, cenário), os processos não emitem uma escrita por hora. Em vez disso, cada processo acumula em memória todos os registros horários pertencentes ao mesmo estágio e cenário e, ao término do cálculo, realiza uma única escrita agregada cobrindo o bloco completo de horas. Essa estratégia reduz o número de chamadas de E/S e aumenta o volume médio por operação, favorecendo o alinhamento dos blocos às fronteiras de *stripe* do Lustre e evitando a contenção de locks associada a escritas pequenas e desalinhadas [25].

O Código 3.1 consolida, em pseudocódigo, a rotina executada por cada *rank* trabalhador ao concluir um cenário, correspondente ao método `escreveRegistro`. Sua organização pode ser lida segundo os três aspectos ortogonais que o padrão MPI-IO atribui às rotinas de acesso a dados — *positioning*, *coordination* e *synchronism* [1, 2] —, somados a uma decisão de *layout* dos blocos gravados. Quanto à *coordination*, a escrita é sempre independente (não coletiva): cada processo emite sua própria operação, em consonância com o modelo *bag-of-tasks* discutido anteriormente. O *positioning* emprega *offset* explícito: o processo deriva a posição de destino da posição lógica do cenário no arquivo (Seção 3.3.2), sem uso de ponteiros implícitos de arquivo. A implementação escolhe o *synchronism* por bloco: ela adota o caminho não bloqueante (`MPI_File_iwrite_at`) para blocos menores que 128 KB e o bloqueante (`MPI_File_write_at`) para os demais. Por fim, a decisão de *layout* determina o que e quanto gravar: o processo serializa os valores dos elementos do sistema elétrico em um registro e acumula em memória os patamares (horas) de um mesmo par (estágio, cenário), emitindo uma única escrita agregada apenas quando processa o último bloco do estágio, o que aumenta o tamanho médio das requisições [15].

As escritas não bloqueantes exigem que o *buffer* de origem permaneça válido

até a conclusão da operação [1, 13]. Para assegurar essa validade sem serializar o fluxo de escrita, a implementação reserva um conjunto de 32 *buffers* pré-alocados e os reutiliza de forma circular (*ring buffer*): antes de reaproveitar uma posição, o processo aguarda, por meio de `MPI_Wait`, a conclusão da escrita anteriormente associada àquele *buffer*. Esse é o único ponto de bloqueio no caminho de escrita e, em regime permanente, o processo raramente o atinge, pois a operação de E/S correspondente costuma já ter sido concluída quando o *buffer* volta a ser necessário. Ao final da simulação, as escritas ainda pendentes são concluídas conjuntamente por meio de `MPI_Waitall`. O tempo despendido nessas esperas — tanto nos `MPI_Wait` realizados ao longo da simulação quanto no `MPI_Waitall` final — foi contabilizado nas medições e mostrou-se desprezível frente ao tempo total de escrita.

```
procedimento escreveRegistro()
    // LAYOUT: serializa os valores do cenario no registro
    para cada elemento i do sistema eletrico:
        registro[i] <- float(dados[i])

    slot <- contador mod TAM_ANEL            // anel de buffers pre-alocados
    // SYNCHRONISM: so reutiliza o slot apos a escrita anterior concluir
    se requisicao[slot] != NULA:
        MPI_Wait(requisicao[slot])

    // LAYOUT: acumula o patamar (hora) no bloco do estagio corrente
    buffer[slot][offset_do_patamar] <- registro

    se patamar = ultimo bloco do estagio:
        // POSITIONING: offset explicito do cenario no arquivo de resultado
        irec     <- posicaoLogica(estagio, cenario, patamar)
        offset   <- irec * tamRegistro
        tamBloco <- tamRegistro * (numero de blocos do estagio)

        // COORDINATION independente + SYNCHRONISM por tamanho de bloco
        se tamBloco < 128 KB:
            MPI_File_iwrite_at(fh, offset, buffer[slot], tamBloco, requisicao[slot])
            contador <- contador + 1
        senao:
            MPI_File_write_at(fh, offset, buffer[slot], tamBloco)

```

Código 3.1: Pseudocódigo da escrita independente executada por cada *rank* trabalhador ao concluir um cenário (método `escreveRegistro`), anotado segundo os aspectos de acesso a dados do MPI-IO — *positioning*, *coordination* e *synchronism* [1, 2] — e a decisão de *layout* dos blocos.

3.3.2 Formato dos resultados

A implementação proposta preserva o formato dos arquivos de resultado utilizado pelo SDDP. Este trabalho adotou essa decisão para manter a compatibilidade com a cadeia de softwares que consome esses arquivos em etapas posteriores do fluxo

de trabalho. Assim, a mudança introduzida pela solução MPI-IO concentra-se na forma como a aplicação grava os dados, sem alterar a estrutura lógica esperada pelos demais programas que dependem das saídas da simulação. Além disso, a manutenção do formato original permite estabelecer uma linha de base para comparações futuras com outras propostas de organização dos arquivos de resultado.

A Figura 3.4 detalha a organização lógica dos arquivos binários de resultado. Cada registro traz, nas três primeiras colunas, o estágio, o cenário e a hora. A partir da quarta coluna, o arquivo armazena os valores associados aos elementos do sistema elétrico. Neste trabalho, um elemento do sistema elétrico corresponde a uma entidade física ou lógica modelada pelo SDDP e representada como uma coluna do arquivo de resultado. Esse elemento pode ser, por exemplo, uma usina hidrelétrica, uma usina térmica, um reservatório, uma barra, uma interligação ou qualquer outro componente do sistema elétrico cuja grandeza a simulação registre em sua saída. Os valores desses elementos variam em função da combinação (estágio, cenário, hora). Essa estrutura permite calcular diretamente a região de escrita de cada cenário no arquivo. Os marcadores laterais indicam os *offsets* de início de cada estágio.



Figura 3.4: Formato lógico dos arquivos binários de resultado do SDDP.

O número de elementos presentes em cada arquivo de resultado pode variar substancialmente de acordo com a base de dados do sistema elétrico analisado, pois cada sistema possui sua própria estrutura de equipamentos e componentes representados no modelo. Como cada elemento corresponde a uma coluna adicional no arquivo, essa variação também altera o tamanho dos blocos de dados gravados.

Por exemplo, considerando um bloco horário associado a um par (estágio, cenário), com 720 horas e valores de 4 bytes, um arquivo com 1 elemento possui 4 colunas — estágio, cenário, hora e o valor do elemento —, totalizando aproximadamente 11,25 KB. Com 10 elementos, o bloco passa a ter 13 colunas e ocupa 36,56 KB. Já com 300 elementos, o bloco possui 303 colunas e ocupa 852,19 KB. Além disso, alguns resultados podem registrar apenas uma única hora, gerando blocos de dados ainda menores. Dessa forma, a estrutura física do sistema elétrico modelado influencia diretamente o volume de dados produzido e, consequentemente, o tamanho das escritas realizadas.

O Lustre aloca cada bloco gravado nos arquivos de resultado em um *stripe* específico de um OST. Quando o volume acumulado ultrapassa o tamanho configurado para o *stripe*, ele aloca um novo *stripe* em outro OST. A atribuição dos *stripes* aos OSTs ocorre de forma circular (*round-robin*), de modo que o sistema distribui fisicamente cada arquivo de resultado entre os múltiplos *stripes* alocados em OSTs distintos. Esse padrão permite paralelizar as operações de escrita sobre um mesmo arquivo, com grau de paralelismo limitado pelo número de OSTs definido na configuração de *striping* associada ao arquivo [7].

Dois parâmetros são centrais nessa configuração. O primeiro é `stripe_count`, que define a quantidade de OSTs usados por arquivo e, portanto, o grau máximo de distribuição física das escritas. O segundo é `stripe_size`, que define o tamanho de cada faixa antes que o arquivo avance para o próximo OST. Valores muito baixos de `stripe_size` podem aumentar a fragmentação das requisições, enquanto valores muito altos podem reduzir a distribuição efetiva entre OSTs. A escolha desses parâmetros deve considerar o tamanho típico dos blocos gravados pelo SDDP e a concorrência esperada entre os *ranks*.

A escolha entre escritas independentes e coletivas favorece a primeira pelo modelo *bag-of-tasks*: cada *rank* persiste seus resultados assim que conclui o cenário recebido. Escritas coletivas poderiam permitir agregação interna pela biblioteca MPI-IO, mas exigiriam que múltiplos *ranks* participassem conjuntamente da mesma chamada de E/S. Nesse contexto, a sincronização adicional poderia bloquear *ranks* que já concluíram sua simulação enquanto aguardam outros cenários ainda em execução. Por esse motivo, a escrita independente é a escolha mais aderente ao padrão de execução avaliado.

3.4 Instrumentação

A avaliação empírica da proposta requer medições detalhadas dos tempos consumidos em cada etapa do ciclo de execução do SDDP. Para isso, este trabalho instrumentou tanto a implementação atual quanto a proposta no módulo de persistência

do SDDP — o mesmo ponto da arquitetura no qual incorporou a solução baseada em MPI-IO. Posicionou os cronômetros imediatamente antes e depois das chamadas instrumentadas utilizando `MPI_Wtime`, primitiva da própria biblioteca MPI cuja resolução é adequada para temporização de operações de E/S e comunicação. Dessa forma, os tempos refletem exatamente o que cada processo observa em sua execução, sem interferência de relógios externos ou de ferramentas de *profiling* fora do espaço de memória da aplicação.

Este trabalho manteve a instrumentação coerente entre as duas implementações, de modo a permitir comparação direta. Cada métrica possui uma definição operacional única, aplicável tanto ao modelo centralizado — em que a transferência dos blocos de dados ocorre por `MPI_Send` e `MPI_Recv` até o *rank* escritor — quanto ao modelo MPI-IO, em que a transferência ocorre por chamadas a `MPI_File_write_at` e `MPI_File_iwrite_at`, conforme o tamanho do bloco de dados.

3.4.1 Registros gerados pela instrumentação

A instrumentação persiste os tempos coletados em arquivos de *log* textuais gerados por *rank*, de forma a evitar contenção sobre um mesmo descritor de arquivo e a permitir consolidação *off-line*. A estratégia de gravação acumula os valores em memória durante a execução e os escreve em disco apenas ao final, fora da janela de medição, de modo a minimizar a perturbação introduzida pela própria instrumentação. A Tabela 3.1 sumariza os arquivos produzidos para cada *rank* p.

Tabela 3.1: Arquivos de *log* gerados pela instrumentação, por *rank* p.

Arquivo	**Conteúdo**
`mpiio-{p}.log`	Um registro por operação de escrita do *rank* p, contendo estágio, cenário, identificador do arquivo, número de sequência, *timestamps* de início e fim da chamada (`MPI_File_write_at` ou `MPI_File_iwrite_at`) e tamanho do bloco em *bytes*.
`mpiio-collective-{p}.log`	Um registro por operação coletiva de E/S, com tipo (`open` ou `close`), nome do arquivo e *timestamps* de início e fim da chamada (`MPI_File_open` ou `MPI_File_close`).
`sddptimer{p}.log`	Temporizadores internos do SDDP, em particular `Simulation` — tempo total da fase de simulação — e `Hourly simulation` — tempo gasto especificamente no trabalho de resolução horária dos cenários, sem chamadas MPI.

A partir desses arquivos, a análise deriva as métricas por agregação. As subseções seguintes descrevem cada métrica em termos das grandezas medidas e do significado físico que carregam para a análise de desempenho.

3.4.2 Tempo de computação

Corresponde ao tempo gasto por cada *rank* no trabalho útil da aplicação — a resolução numérica dos cenários a ele atribuídos —, excluídas as chamadas MPI de coordenação e as operações de E/S. Esse intervalo provém do temporizador `Hourly simulation` do `sddptimer{p}.log`, que delimita estritamente o trecho de código responsável pela simulação horária no SDDP.

Em um cenário de *strong scaling*, espera-se que esse tempo decresça aproximadamente de forma linear com o número de nós, dado que cada nó passa a processar uma fração menor do total de cenários. Desvios sistemáticos dessa tendência indicariam contenção indireta sobre recursos compartilhados (memória, *cache* ou rede) ou desbalanceamento entre os *ranks*, mas não devem ser sensíveis ao mecanismo de E/S adotado. O tempo de computação serve, portanto, como referência estável contra a qual os tempos de comunicação, de E/S e de coordenação podem ser comparados.

3.4.3 Tempo de `MPI_File_open`/`MPI_File_close`

Reúne os tempos despendidos em chamadas MPI que não correspondem à transferência dos blocos de dados em si, mas sim ao estabelecimento de pontos de coordenação entre processos. Essa categoria inclui as operações coletivas de abertura e fechamento dos arquivos de resultado (`MPI_File_open` e `MPI_File_close`), registradas no `mpiio-collective-{p}.log`. Por se tratar de operações coletivas exigidas pela semântica da biblioteca MPI-IO, são inerentes à implementação proposta e ausentes na implementação atual, que opera apenas com primitivas ponto a ponto.

Essa métrica é particularmente relevante para a arquitetura proposta, pois revela o custo embutido nas operações coletivas, que tende a crescer com o número de *ranks* participantes. Em escala, esse custo pode emergir como gargalo adicional, mesmo quando o tempo de E/S por *rank* se reduz [12]. Capturá-lo separadamente permite distinguir o ganho líquido da paralelização das escritas do custo adicional imposto pela própria coordenação coletiva.

3.4.4 Tempo total de E/S

Corresponde ao tempo total que cada *rank* dedica às operações associadas à persistência dos blocos de dados, calculado pela soma dos intervalos $T_e^{\text{io}} - T_s^{\text{io}}$ registrados no `mpiio-{p}.log`. Na implementação MPI-IO, esse tempo corresponde às chama-

das `MPI_File_write_at` (síncronas, para blocos $\geq$ 128 KB) e `MPI_File_iwrite_at` (assíncronas, para blocos $<$ 128 KB). Na implementação atual, embora o *rank* escritor concentre toda a escrita em disco, a contabilização inclui o tempo gasto pelos *ranks* trabalhadores nas chamadas `MPI_Send` responsáveis pelo encaminhamento dos blocos, de modo a tornar as duas instrumentações diretamente comparáveis.

A métrica caracteriza, de forma agregada, o custo computacional que a E/S representa para a aplicação, independentemente de o trabalho estar concentrado em um único processo ou distribuído entre todos. Diferenças entre as duas implementações nessa métrica refletem o impacto direto da paralelização das escritas sobre o caminho de persistência.

3.4.5 Tempo de bloqueio na transferência dos blocos de dados

Corresponde ao tempo total, ao longo da execução, em que o *rank* permaneceu impedido de progredir em razão das operações de envio dos blocos de dados para persistência. Em termos operacionais, é a soma dos intervalos das chamadas de transferência registrados no `mpiio-{p}.log`: na implementação proposta, as chamadas síncronas `MPI_File_write_at`, em que o bloqueio coincide com a duração integral da chamada, e as chamadas assíncronas `MPI_File_iwrite_at`, em que o bloqueio reduz-se ao tempo de despacho da operação, tipicamente desprezível. Na implementação atual, em que as transferências ocorrem por `MPI_Send` até o *rank* escritor, o intervalo bloqueante é a própria duração da chamada `MPI_Send`.

O tempo de bloqueio, ao agregar todas as parcelas perceptíveis como atraso pela aplicação, é o que efetivamente limita a sobreposição entre a computação e as operações de escrita. Para blocos pequenos, predominam custos fixos de iniciação; para blocos grandes, predomina o tempo efetivo de transferência. Em uma execução bem dimensionada, espera-se que essa parcela seja baixa, indicando que a estratégia assíncrona e o paralelismo de escritas conseguiram mascarar a latência do subsistema de armazenamento. Crescimentos sistemáticos do tempo de bloqueio em função do número de *ranks* sugerem saturação dos recursos do sistema de arquivos ou contenção entre escritas concorrentes.

3.4.6 Banda agregada média percebida pela aplicação

Este trabalho define como *banda agregada percebida pela aplicação* a métrica que sintetiza, em uma única grandeza, quanto da capacidade de E/S disponível a aplicação SDDP efetivamente consegue aproveitar sob seu modelo de execução. O cálculo parte dos registros do `mpiio-{p}.log`, particionando a fase de simulação em janelas de tempo de duração fixa Δ. Para cada janela w, a métrica corresponde à razão entre o volume total de dados gravados por todos os *ranks* naquele intervalo e o

tempo de E/S acumulado pelo *rank* mais lento da janela:

$$B_w \;=\; \frac{\displaystyle\sum_{i=1}^{P} V_{i,w}}{\displaystyle\max_{i=1,\ldots,P} T_{i,w}^{\mathrm{E/S}}},$$

em que $V_{i,w}$ é o volume gravado pelo *rank* i na janela w, $T_{i,w}^{\mathrm{E/S}}$ é o tempo de E/S acumulado por esse *rank* dentro da mesma janela e P é o número de processos. O denominador adota o tempo do processo mais lento, em consonância com o comportamento síncrono da aplicação ao final de cada janela: o avanço para a próxima etapa só é possível quando todos os *ranks* concluem suas escritas. O valor reportado para uma execução é a média aritmética de B_w ao longo das janelas que cobrem a fase de simulação, e a banda agregada média associada a uma configuração de nós corresponde à média dessa grandeza ao longo das cinco repetições realizadas para cada configuração, acompanhada do respectivo desvio padrão.

Relação entre banda percebida e tempo de bloqueio. A banda percebida depende do tempo de bloqueio das operações de escrita, conforme definido na Seção 3.4.5. Para escritas síncronas (`MPI_File_write_at`), o intervalo registrado no `mpiio-{p}.log` corresponde à duração integral da chamada, durante a qual o *rank* fica impedido de prosseguir. Para escritas assíncronas (`MPI_File_iwrite_at`), o intervalo registrado refere-se apenas ao despacho da operação — tipicamente desprezível —, de modo que escritas assíncronas bem-sucedidas não inflam o denominador. Em consequência, a métrica captura a parcela de sobrecarga que se traduz efetivamente em bloqueio do caminho de execução. Uma estratégia que reduza esse tempo de bloqueio — seja por paralelizar escritas em múltiplos OSTs, seja por mascará-las com chamadas assíncronas — eleva a banda percebida, tornando-a adequada para a comparação direta entre as duas implementações investigadas.

Capítulo 4

Avaliações

Este capítulo apresenta a avaliação comparativa entre a implementação atual de entrada e saída (E/S) e a implementação proposta, com o objetivo de investigar ganhos de desempenho e melhorias de escalabilidade, principalmente em ambientes de computação em nuvem. A análise concentra-se em métricas de tempo de execução, tempo de comunicação, largura de banda e tempo de bloqueio das operações de E/S, considerando o impacto das estratégias adotadas para operações paralelas de E/S em cenários de alto desempenho [26, 27].

A base de dados utilizada nos experimentos corresponde ao PMO/PAR de outubro de 2023. Para a etapa de simulação horária, que constitui o foco principal deste estudo, este trabalho considerou 1.536 cenários em ambos os experimentos (AWS e SDumont), com três estágios mensais.

Para cada configuração avaliada, este trabalho executou cinco repetições; os valores reportados nas tabelas e figuras deste capítulo correspondem à média e ao desvio padrão dessas amostras.

Como cada *rank* MPI apresentou consumo de memória residente superior a 8 GB, este trabalho optou, em ambos os experimentos, por mapear apenas um *rank* por núcleo físico — isto é, sem o uso do *hyper-threading*. A memória total disponível por nó impõe essa decisão: com 384 GB por nó nos dois ambientes, ativar *hyper-threading* e dobrar o número de *ranks* ultrapassaria a memória física agregada, inviabilizando a execução. Este trabalho fixou, portanto, o número de *ranks* por nó no número de núcleos físicos da instância correspondente.

Os experimentos conduzidos neste trabalho caracterizam um cenário de *strong scaling*, no qual o tamanho do problema permanece fixo enquanto o número de nós computacionais aumenta progressivamente [28]. Nesse contexto, o acréscimo de recursos implica na redução do volume de trabalho por nó, o que, idealmente, faria o tempo total de execução diminuir de forma aproximadamente linear com o aumento do paralelismo. No entanto, essa tendência teórica pode não se concretizar na prática devido à presença de gargalos sistêmicos, especialmente aqueles relacionados

às operações de entrada e saída (E/S) e à comunicação entre processos. À medida que o número de nós cresce, a intensificação da concorrência por recursos compartilhados — como o sistema de arquivos e a rede — pode introduzir sobrecargas adicionais, limitando a eficiência do escalonamento e reduzindo os ganhos esperados de desempenho.

Este trabalho considerou o tamanho dos blocos de dados um fator determinante para a análise de desempenho de E/S paralela. As Figuras 4.1 e 4.2 caracterizam o *workload* de escrita sob duas óticas complementares — a frequência das operações e o volume de dados gravado —, ambas por faixa de tamanho de bloco.

A Figura 4.1 apresenta a distribuição agregada do número de escritas por faixa de tamanho. Observa-se que blocos de dados pequenos dominam a distribuição: 1.801.728 blocos, correspondentes a 80,1% do total, possuem tamanho de até 1 KB. As faixas intermediárias — até 32 KB, 64 KB e 128 KB — somam, em conjunto, 152.082 blocos (6,8%), enquanto os blocos de até 1 MB respondem por 267.264 escritas (11,9%) e os de até 50 MB por 27.648 (1,2%).

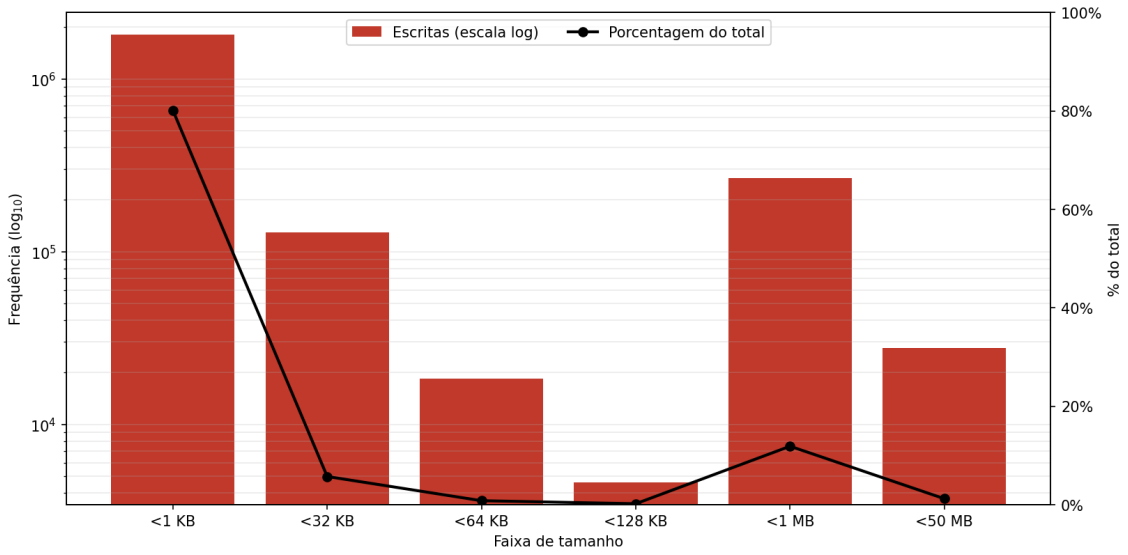


Figura 4.1: Distribuição agregada do número de escritas por faixa de tamanho de bloco no experimento (frequência em escala logarítmica; linha: percentual do total).

A Figura 4.2 revela um quadro oposto quando o critério passa a ser o volume de dados gravado: embora numerosos, os blocos pequenos contribuem com parcela desprezível do total escrito, ao passo que praticamente todo o volume se concentra nas duas maiores faixas — aproximadamente 117 GB nos blocos de até 1 MB e cerca de 446 GB nos de até 50 MB, de um total de cerca de 567 GB gravados. Em outras palavras, as escritas pequenas predominam em quantidade, mas as grandes predominam em bytes transferidos.

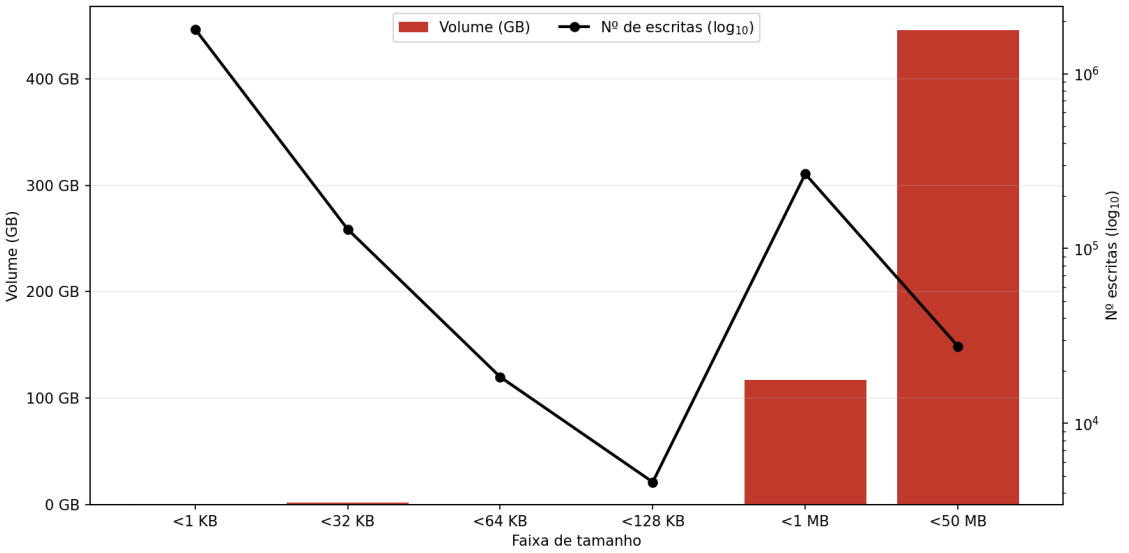


Figura 4.2: Volume total de dados gravado por faixa de tamanho de bloco no experimento (barras em GB; linha: número de escritas em escala logarítmica).

Esse contraste entre frequência e volume pode comprometer a escalabilidade: a alta frequência de operações de granularidade fina tende a aumentar a sobrecarga de metadados, enquanto o volume concentrado nos blocos maiores impõe a maior demanda sobre a largura de banda do sistema de arquivos [26, 27]. Adicionalmente, este trabalho avaliará o tempo de envio dos blocos de dados agrupados por faixas de tamanho, de forma a analisar o impacto da granularidade das operações no desempenho da comunicação e da E/S.

A distribuição observada nas Figuras 4.1 e 4.2 está diretamente associada à organização dos arquivos de resultado produzidos pelo SDDP. Nos experimentos, este trabalho gerou 117 arquivos de resultado: 104 no formato horário e 13 no formato diário. Nos arquivos horários, a aplicação acumula em memória os registros de um par $(\text{estágio}, \text{cenário})$ e os grava em uma única operação agregada, conforme descrito na Seção 3.3.2; já nos arquivos diários, a aplicação grava cada dia do mês individualmente, sem agregação entre dias. Essa diferença de granularidade explica o perfil observado: os 13 arquivos diários respondem pela grande maioria dos blocos pequenos (até 1 KB), enquanto os blocos médios e grandes provêm dos 104 arquivos horários.

4.1 Experimento AWS

Este trabalho configurou o Experimento AWS na nuvem da Amazon Web Services (AWS), utilizando 32 nós da instância do tipo r7i.12xlarge. Essa instância é

baseada na quarta geração de processadores Intel Xeon Scalable (Sapphire Rapids 8488C), operando a 3,2 GHz, e disponibiliza 24 núcleos físicos com suporte a *hyper-threading*, totalizando 48 processadores lógicos. O sistema conta ainda com 384 GB de memória DDR5, que oferece maior largura de banda em relação às gerações anteriores, e a placa de rede *Elastic Network Adapter* (ENA), baseada em Ethernet, com capacidade de até 18,75 Gbps, projetada para cargas de trabalho intensivas em comunicação [21]. Essa configuração enquadra-se na categoria de instâncias otimizadas para memória da AWS (*memory-optimized*), adequada a aplicações que combinam elevada demanda de processamento paralelo com grandes volumes de dados em memória — perfil característico do SDDP, em que cada *rank* MPI chega a consumir mais de 8 GB de memória residente. O experimento utilizou a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre versão 2.12 de 12 TB, com 1 MDS de 350 GB e 10 OSTs de 1,165 TB. Este trabalho definiu o *stripe count* como 10, com *stripes* de 1 MB, disponibilizado por meio do serviço Amazon FSx for Lustre. Pelos motivos discutidos na introdução do capítulo, este trabalho alocou um *rank* por núcleo físico, totalizando 24 *ranks* MPI por nó, que processam cenários em paralelo segundo o padrão *bag-of-tasks* descrito na Seção 2.5 [29, 30].

4.1.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento AWS. A Tabela 4.1 sumariza os resultados obtidos com a base de dados PMO/PAR de outubro de 2023, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

Observa-se que o tempo total de execução cai com o aumento do número de nós até 16 nós, passando de aproximadamente 8 971 s em 2 nós para 1 874 s em 16 nós, mas volta a crescer para 2 013 s em 32 nós — caracterizando o regime saturado típico de um cenário de *strong scaling* limitado pela E/S. A banda agregada degrada-se de 60,5 Gb/s em 2 nós para 3,4 Gb/s em 32 nós (mais de uma ordem de grandeza), e o percentual do tempo dedicado à E/S cresce de 0,34% em 2 nós para 49,51% em 32 nós. Esses números são coerentes com o diagnóstico de que a serialização no nó escritor se torna ponto de contenção à medida que aumenta o número de processos emissores concorrentes. Vale notar que a banda percebida em pequena escala (60,5 Gb/s em 2 nós) excede a capacidade nominal do adaptador de rede da r7i.12xlarge (18,75 Gb/s). Esse aparente excesso é consistente com a definição da métrica (Seção 3.4.6): em 2 nós, o sistema absorve os blocos com bloqueio mínimo nos *buffers* da pilha MPI e do sistema operacional, e o denominador da banda percebida fica reduzido. Conforme cresce o número de *ranks* emissores, a serialização no nó escritor satura, os bloqueios se acumulam e a banda percebida converge para

valores muito inferiores à capacidade nominal do adaptador — exatamente o regime observado a partir de 8 nós.

Soma-se a esse efeito a heterogeneidade entre envios locais e remotos. Na implementação atual, os *ranks* trabalhadores transferem seus blocos ao *rank* escritor por meio de `MPI_Send`. Mesmo sendo síncrono, esse envio, quando emissor e receptor residem no mesmo nó, ocorre por canais de memória compartilhada da biblioteca MPI e apresenta tempo de bloqueio significativamente inferior ao dos envios remotos, que precisam atravessar o NIC do nó emissor e o do nó escritor. Esse efeito é particularmente acentuado na configuração de 2 nós, em que cerca de metade dos *ranks* trabalhadores reside no mesmo nó do escritor e produz envios locais — inflando ainda mais a banda percebida nessa escala. À medida que cresce o número de nós, a fração de envios locais diminui rapidamente (aproximadamente um quarto em 4 nós, um oitavo em 8 nós e abaixo de 6% em 16 nós), e o comportamento da métrica passa a refletir predominantemente o tempo de bloqueio das transferências remotas, governadas pela serialização no nó escritor.

Na Tabela 4.1, o tempo total corresponde ao tempo de execução medido, que inclui computação, E/S e comunicação. A tabela apresenta separadamente os tempos de I/O e comunicação.

Tabela 4.1: Desempenho do Experimento AWS com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, *speedup*, eficiência e percentual de I/O. Este trabalho calcula a eficiência em relação à base de 2 nós.

Nós	Banda percebida (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	$60{,}51 \pm 5{,}20$	$30{,}85 \pm 13{,}23$	0,34%	$177{,}51 \pm 29{,}73$	$8\,971{,}23 \pm 348{,}60$	1,00×	100,0%
4	$51{,}28 \pm 2{,}82$	$78{,}23 \pm 27{,}79$	1,62%	$212{,}76 \pm 19{,}33$	$4\,823{,}20 \pm 53{,}08$	1,86×	93,0%
8	$18{,}94 \pm 3{,}76$	$201{,}67 \pm 53{,}32$	7,64%	$174{,}04 \pm 12{,}90$	$2\,639{,}18 \pm 18{,}34$	3,40×	85,0%
16	$4{,}52 \pm 0{,}33$	$527{,}94 \pm 53{,}07$	28,18%	$167{,}77 \pm 7{,}07$	$1\,873{,}56 \pm 56{,}94$	4,79×	59,9%
32	$3{,}37 \pm 0{,}10$	$996{,}57 \pm 166{,}98$	49,51%	$304{,}96 \pm 3{,}83$	$2\,013{,}06 \pm 29{,}10$	4,46×	27,9%

4.1.2 Avaliação da implementação MPI-IO com escritas independentes

Esta subseção analisa o desempenho da arquitetura proposta, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre, no Experimento AWS. Nesta abordagem cada processo MPI escreve seus próprios dados diretamente no FSx for Lustre, eliminando o ponto de contenção associado a um único processo escritor e explorando o paralelismo dos múltiplos OSTs.

A Tabela 4.2 consolida os resultados obtidos com a implementação MPI-IO no Experimento AWS. Assim como na tabela anterior, o tempo total corresponde ao

tempo de execução medido, que inclui computação, E/S, comunicação e `MPI_File_open`/`MPI_File_close`. A tabela apresenta separadamente os tempos de I/O, comunicação e `MPI_File_open`/`MPI_File_close`.

Tabela 4.2: Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, coordenação, tempo total, *speedup*, eficiência e percentual de I/O. Este trabalho calcula a eficiência em relação à base de 2 nós.

Nós	Banda percebida (Gb/s)	I/O (s)	% I/O	Comunicação (s)	`MPI_File_open`/ `MPI_File_close` (s)	Total (s)	Speedup	Eficiência (%)
2	$57{,}04 \pm 8{,}47$	$18{,}98 \pm 5{,}55$	0,21%	$188{,}24 \pm 25{,}66$	$42{,}48 \pm 7{,}69$	$8\,858{,}91 \pm 111{,}44$	1,00×	100,0%
4	$106{,}04 \pm 19{,}42$	$12{,}79 \pm 3{,}50$	0,25%	$233{,}70 \pm 24{,}23$	$51{,}23 \pm 15{,}15$	$5\,127{,}24 \pm 107{,}53$	1,73×	86,4%
8	$167{,}03 \pm 40{,}19$	$10{,}75 \pm 6{,}25$	0,41%	$185{,}30 \pm 19{,}97$	$69{,}82 \pm 4{,}68$	$2\,599{,}75 \pm 37{,}14$	3,41×	85,2%
16	$274{,}24 \pm 15{,}33$	$7{,}47 \pm 1{,}78$	0,48%	$233{,}71 \pm 5{,}14$	$83{,}53 \pm 4{,}16$	$1\,558{,}33 \pm 9{,}39$	5,68×	71,1%
32	$380{,}30 \pm 30{,}65$	$8{,}96 \pm 2{,}46$	0,70%	$235{,}99 \pm 9{,}30$	$147{,}26 \pm 7{,}32$	$1\,274{,}25 \pm 23{,}90$	6,95×	43,5%

4.1.3 Análise comparativa

As figuras a seguir consolidam visualmente os resultados das duas subseções anteriores, apresentando, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações.

A Tabela 4.3 resume os principais indicadores comparativos do Experimento AWS. A melhoria no tempo total é pequena em 2 nós, torna-se negativa em 4 nós, mas passa a ser positiva a partir de 8 nós e cresce nas maiores escalas: 1,5% em 8 nós, 16,8% em 16 nós e 36,7% em 32 nós. A comparação de banda evidencia o principal ganho da arquitetura proposta: a MPI-IO passa de desempenho ligeiramente inferior em 2 nós para 2,07× a banda da implementação atual em 4 nós, 8,82× em 8 nós, 60,66× em 16 nós e 113,00× em 32 nós. A redução do tempo de E/S também cresce com a escala: 38,5% em 2 nós, 83,7% em 4 nós, 94,7% em 8 nós, 98,6% em 16 nós e 99,1% em 32 nós.

Tabela 4.3: Resumo comparativo entre a implementação atual e MPI-IO no Experimento AWS. Este trabalho calcula a melhoria no total e a melhoria de I/O em relação à implementação atual; valores negativos indicam piora.

Nós	Total atual (s)	Total MPI-IO (s)	Melhoria no total (%)	Banda percebida atual (Gb/s)	Banda percebida MPI-IO (Gb/s)	Ganho de banda perc.	I/O atual (s)	I/O MPI-IO (s)	Melhoria I/O (%)
2	8 971,23	8 858,91	1,3%	60,51	57,04	0,94×	30,85	18,98	38,5%
4	4 823,20	5 127,24	−6,3%	51,28	106,04	2,07×	78,23	12,79	83,7%
8	2 639,18	2 599,75	1,5%	18,94	167,03	8,82×	201,67	10,75	94,7%
16	1 873,56	1 558,33	16,8%	4,52	274,24	60,66×	527,94	7,47	98,6%
32	2 013,06	1 274,25	36,7%	3,37	380,30	113,00×	996,57	8,96	99,1%

A Figura 4.3 apresenta a evolução da banda agregada média percebida pela aplicação no Experimento AWS. As duas curvas exibem comportamentos qualitativamente opostos: a implementação atual parte de aproximadamente 60,5 Gb/s em 2 nós e degrada-se de forma acentuada com o aumento do paralelismo, atingindo 18,9 Gb/s em 8 nós, 4,5 Gb/s em 16 nós e 3,4 Gb/s em 32 nós — uma redução de mais de uma ordem de grandeza. Em contraste, a implementação MPI-IO parte de 57,0 Gb/s em 2 nós e cresce monotonicamente, alcançando 106,0 Gb/s em 4 nós, 167,0 Gb/s em 8 nós, 274,2 Gb/s em 16 nós e 380,3 Gb/s em 32 nós.

Em 2 nós, a banda agregada da implementação atual é cerca de 6% maior que a da implementação MPI-IO. Esse comportamento é coerente com o fato de que, em pequena escala, o gargalo do processo escritor único ainda não se manifesta, e o custo fixo da coordenação da E/S paralela tem maior peso relativo. A partir de 4 nós, entretanto, a implementação MPI-IO já supera a atual. Em 8 nós, a banda agregada é aproximadamente 8,8× maior; em 16 nós, cerca de 60,7× maior; e, em 32 nós, aproximadamente 113,0× maior. Esse resultado indica que a arquitetura proposta explora a infraestrutura de armazenamento em uma escala completamente diferente da arquitetura baseada em escritor único.

Essa diferença de comportamento decorre diretamente da eliminação do processo escritor único. Na arquitetura atual, todas as escritas convergem para um único nó escritor, cuja serialização rapidamente se torna ponto de contenção e limita o *throughput* global — explicando a queda contínua da banda observada. Na abordagem MPI-IO, a aplicação distribui as operações de escrita entre os processos MPI, permitindo enviar simultaneamente múltiplos fluxos de dados ao sistema de arquivos Lustre, explorando o paralelismo proporcionado pelos múltiplos OSTs. Como consequência, a banda agregada cresce de forma consistente com o número de nós, demonstrando que o sistema de armazenamento ainda não atingiu seu limite nominal na maior configuração avaliada. Dessa forma, a figura evidencia que a adoção de MPI-IO não apenas elimina o gargalo da arquitetura atual, mas também permite um uso significativamente mais eficiente da largura de banda disponível, tornando a aplicação adequada para execuções em larga escala.

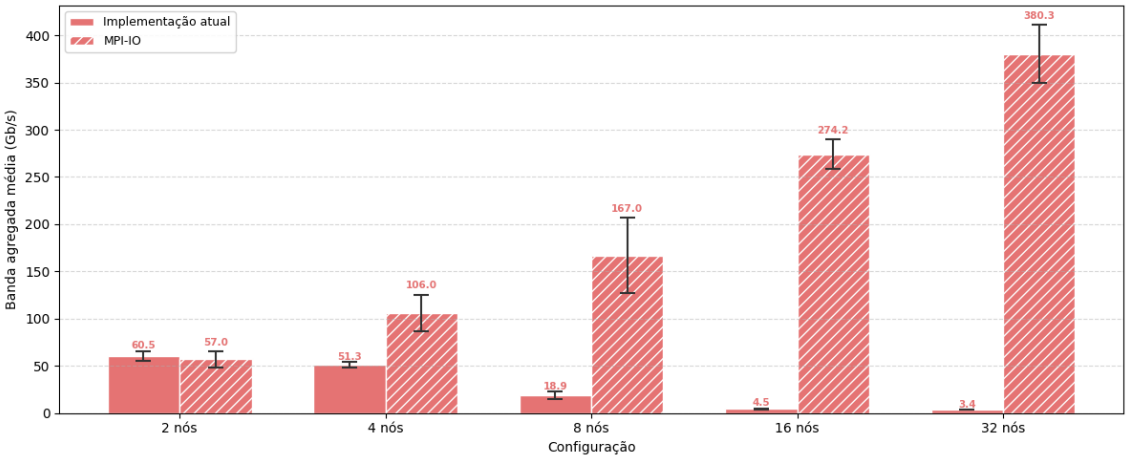


Figura 4.3: Banda agregada média percebida pela aplicação no Experimento AWS.

A Figura 4.4 apresenta a evolução do tempo total de execução no Experimento AWS, contrastando a implementação atual e a implementação MPI-IO sobre o Lustre. As barras de erro representam apenas o desvio padrão do tempo de execução total, e não desvios individuais das parcelas internas. Nas configurações de menor escala, em que a E/S não é o componente dominante, as duas curvas evoluem de forma próxima: em 2 nós, $8{,}97 \times 10^3$ s no modelo atual contra $8{,}86 \times 10^3$ s no modelo MPI-IO; em 4 nós, entretanto, a versão MPI-IO apresenta $5{,}13 \times 10^3$ s contra $4{,}82 \times 10^3$ s da implementação atual. Esse comportamento é coerente, uma vez que, em pequena escala, a computação domina o tempo de execução e o custo da coordenação ainda tem maior peso relativo.

A divergência entre as duas curvas torna-se evidente a partir de 8 nós e se acentua de forma marcante nas configurações de maior porte. Enquanto a implementação atual apresenta um regime de saturação — com o tempo de execução passando de $2{,}64 \times 10^3$ s em 8 nós para $1{,}87 \times 10^3$ s em 16 nós e voltando a crescer para $2{,}01 \times 10^3$ s em 32 nós —, a implementação MPI-IO mantém uma redução consistente do tempo total, atingindo $2{,}60 \times 10^3$ s em 8 nós, $1{,}56 \times 10^3$ s em 16 nós e $1{,}27 \times 10^3$ s em 32 nós. Em termos relativos, isso corresponde a uma redução do tempo total de aproximadamente 1,5% em 8 nós, 16,8% em 16 nós e 36,7% em 32 nós em relação ao modelo atual.

A figura também evidencia que, na implementação atual, o crescimento do número de nós deixa de produzir ganho de desempenho a partir de 16 nós e passa a ser, na prática, contraproducente em 32 nós — comportamento característico de regime degradado sob *strong scaling*, no qual a contenção do processo escritor mais do que compensa os ganhos de paralelismo computacional. Em contraste, a curva MPI-IO permanece monotonicamente decrescente em todo o intervalo avaliado, indicando que a infraestrutura de armazenamento ainda não atingiu sua capacidade

limite mesmo na maior configuração testada.

Cabe observar que, na implementação MPI-IO, o tempo de E/S torna-se uma fração pequena do tempo total em todas as escalas, permanecendo próximo ou abaixo de 1% a partir de 4 nós. Na implementação atual, por outro lado, a fração de E/S cresce de 7,6% em 8 nós para 28,2% em 16 nós e 49,5% em 32 nós. Na implementação MPI-IO, as chamadas `MPI_File_open`/`MPI_File_close` passam a ocupar parte do tempo de simulação: essa parcela cresce de 0,5% em 2 nós para 1,0% em 4 nós, 2,7% em 8 nós, 5,4% em 16 nós e 11,6% em 32 nós. Esse crescimento percentual, contudo, reflete sobretudo a redução do tempo de simulação com o aumento do paralelismo e não um aumento da carga útil tratada por essas chamadas. Como a aplicação executa `MPI_File_open`/`MPI_File_close` uma única vez por execução — na abertura e no fechamento dos arquivos, no início e no final —, esse custo **independe do tamanho da carga**: não cresce com o número de cenários nem com o número de estágios resolvidos, dependendo apenas do número de nós que participam da operação coletiva. É exatamente esse comportamento que se observa nos dados: para um mesmo tamanho de problema, a parcela absoluta cresce de 42 s em 2 nós para 147 s em 32 nós. Os experimentos aqui apresentados consideram apenas três estágios mensais, o que mantém o tempo total de simulação relativamente curto e infla a fração percentual da `MPI_File_open`/`MPI_File_close`; em execuções de operação programada típicas do SIN, com horizonte de vários anos e número de cenários muito maior, o custo absoluto dessas chamadas, para uma dada configuração de nós, permanece o mesmo enquanto o tempo total cresce em ordem de magnitude, diluindo a parcela de coordenação para níveis desprezíveis. A partir de 8 nós, e mesmo sob essa inflação relativa do custo de coordenação, a implementação MPI-IO já apresenta tempo total inferior ao da implementação atual.

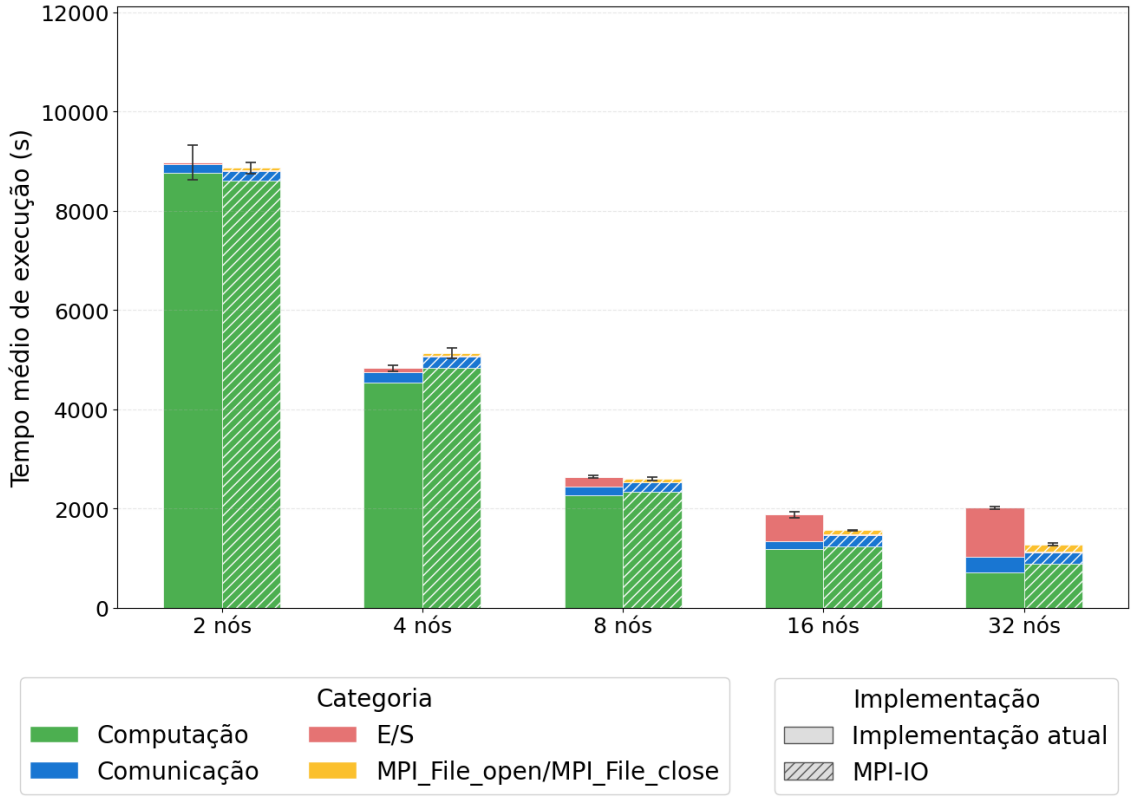


Figura 4.4: Tempo de execução no Experimento AWS.

A Figura 4.5 apresenta o tempo médio de bloqueio das operações de E/S, em segundos, em função do tamanho dos blocos de dados no Experimento AWS, em escala logarítmica. A comparação mostra que, para blocos de dados muito pequenos, a implementação atual apresenta tempos menores, pois o custo fixo das chamadas MPI-IO se torna dominante em relação ao volume transferido. Esse efeito, contudo, ocorre em uma faixa de tempo baixa, da ordem de microssegundos a poucos milissegundos, e não determina o tempo total da aplicação.

Para blocos de dados médios e grandes, que concentram o impacto de E/S nas maiores escalas, o comportamento se inverte. Em 16 e 32 nós, a implementação atual passa a apresentar tempos de bloqueio elevados nas classes de maior tamanho, chegando à ordem de segundos, enquanto a implementação MPI-IO permanece na ordem de centésimos ou décimos de segundo. Assim, a figura reforça o diagnóstico das Figuras 4.3 e 4.4: a abordagem com escritor único é competitiva apenas em pequena escala ou para blocos de dados muito pequenos, mas perde escalabilidade quando o volume de dados e a concorrência entre processos aumentam.

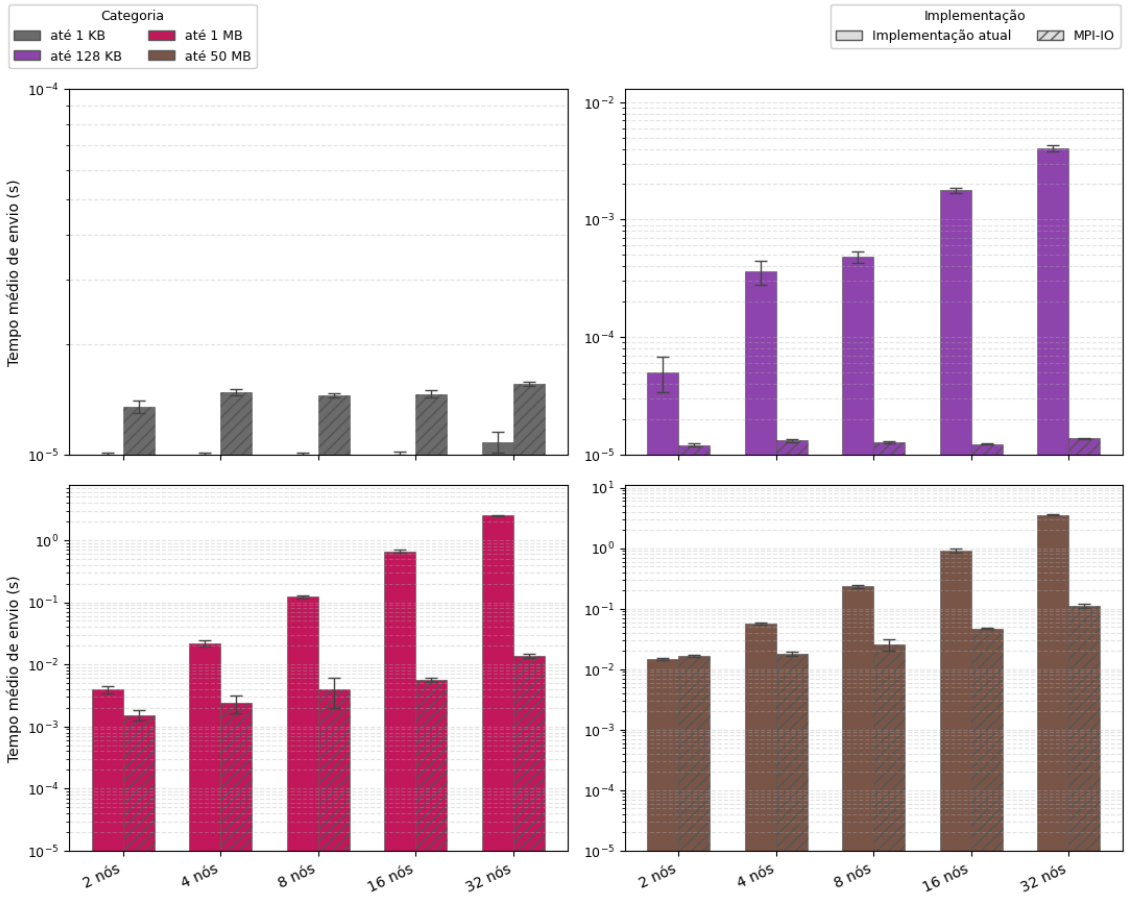


Figura 4.5: Tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento AWS (escala logarítmica).

4.1.4 Avaliação do impacto do número de OSTs

Esta subseção tem como objetivo avaliar, no Experimento AWS, o impacto do número de *Object Storage Targets* (OSTs) no desempenho das operações de E/S no sistema de arquivos Lustre. Em ambientes HPC, a quantidade de OSTs está diretamente relacionada ao grau de paralelismo de E/S disponível, uma vez que o Lustre pode distribuir os dados entre múltiplos dispositivos de armazenamento, permitindo acessos concorrentes por diferentes processos.

A configuração de 10 OSTs aqui considerada é a mesma utilizada no Experimento AWS principal (Subseção 4.1.2, Tabela 4.2); este trabalho obteve a configuração de 4 OSTs em rodada dedicada, com todos os demais parâmetros mantidos constantes — base de dados PMO/PAR de outubro de 2023, 1.536 cenários, três estágios mensais, 24 *ranks* MPI por nó e mesma instrumentação MPI-IO. Os valores absolutos das duas configurações são, portanto, diretamente comparáveis, e o foco da análise é o efeito isolado do número de OSTs sobre o desempenho das operações de E/S.

Para esta avaliação, este trabalho realizou experimentos com a abordagem MPI-IO, variando a quantidade de OSTs disponíveis no FSx for Lustre, comparando configurações com 4 OSTs e 10 OSTs para execuções com 16 e 32 nós de cálculo. Este trabalho manteve constantes as demais características do experimento, incluindo o volume de dados e o padrão de acesso, de modo a isolar o efeito da infraestrutura de armazenamento.

A Figura 4.6 apresenta o tempo médio de envio por classe de tamanho das mensagens. Cada subfigura corresponde a uma classe de tamanho, enquanto as barras comparam as configurações MPI-IO com 4 e 10 OSTs para 16 e 32 nós de cálculo. Em 16 nós, as classes pequenas apresentam tempos da mesma ordem de grandeza nas duas configurações: 0,014 ms com 4 OSTs contra 0,015 ms com 10 OSTs nas mensagens de até 1 KB, e 0,014 ms contra 0,012 ms nas mensagens de até 128 KB. Nas classes que concentram maior volume de dados, o ganho do paralelismo de armazenamento se manifesta com força: o tempo cai de 82,6 ms para 5,7 ms nas mensagens de até 1 MB (aproximadamente 14,5× mais rápido) e de 524 ms para 47,1 ms nas mensagens de até 50 MB (aproximadamente 11,1×).

O efeito é ainda mais pronunciado em 32 nós, quando a maior concorrência entre processos amplia a diferença entre as duas configurações. As classes pequenas mantêm-se com tempos próximos — 0,018 ms com 4 OSTs contra 0,016 ms com 10 OSTs nas mensagens de até 1 KB, e 0,018 ms contra 0,014 ms nas mensagens de até 128 KB —, mas o ganho nas classes maiores se acentua: o tempo médio cai de 309 ms para 13,8 ms nas mensagens de até 1 MB (cerca de 22,4× mais rápido) e de 1,89 s para 113,5 ms nas mensagens de até 50 MB (cerca de 16,6×). Esses resultados mostram que a redução do número de OSTs aumenta a contenção no acesso aos dados principalmente nas classes de maior volume, sobretudo quando o número de nós de cálculo cresce. Embora a aplicação realize a escrita em paralelo por meio de MPI-IO, a disponibilidade de menos alvos de armazenamento limita o paralelismo efetivo do Lustre e aumenta o tempo observado nas operações de envio dessas classes.

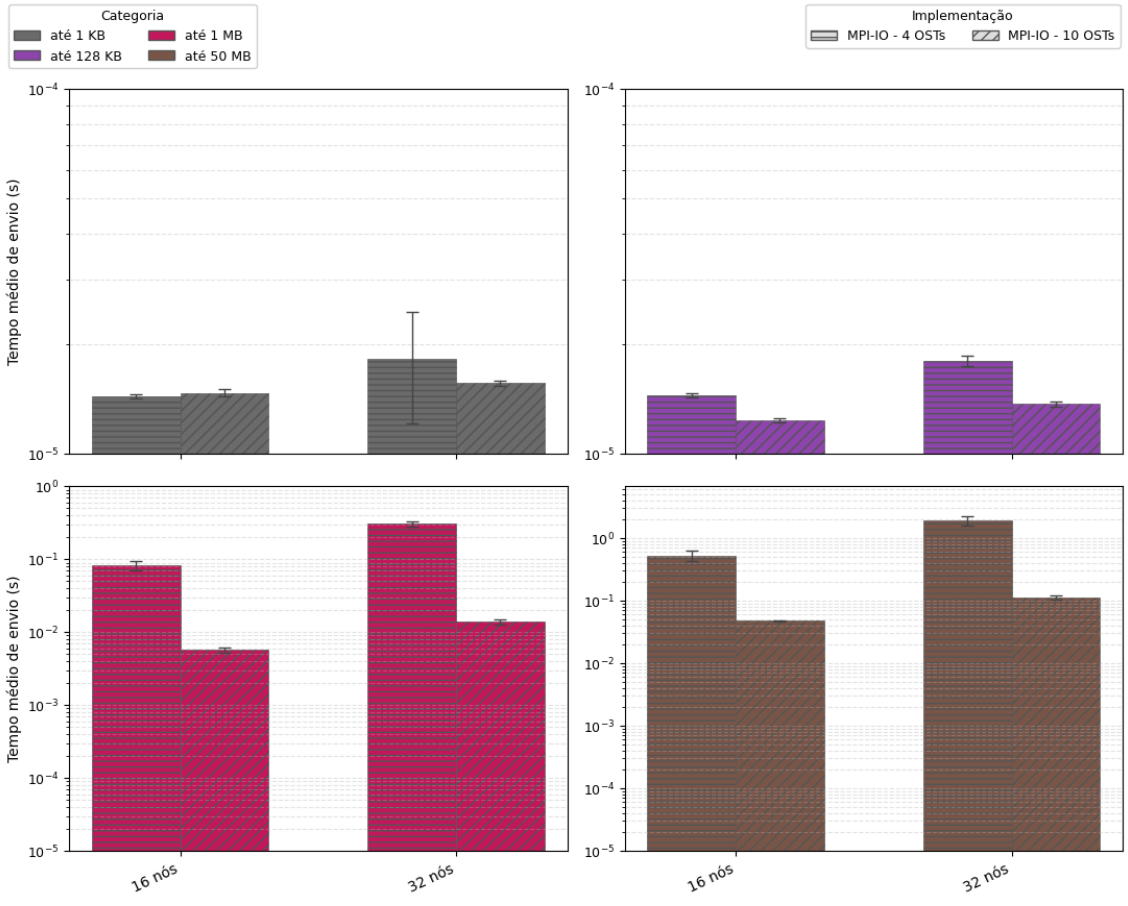


Figura 4.6: Tempo médio de envio por classe de tamanho das mensagens para configurações MPI-IO com 4 e 10 OSTs.

A Figura 4.7 complementa essa análise ao apresentar a banda agregada média percebida pela aplicação em cada configuração. Com 16 nós, a configuração com 10 OSTs atinge 274,2 Gb/s, contra 49,4 Gb/s com 4 OSTs, ganho de aproximadamente 5,6×. Ao passar de 16 para 32 nós, as configurações exibem tendências opostas: com 10 OSTs, a banda cresce para 380,3 Gb/s, enquanto, com 4 OSTs, cai para 16,1 Gb/s. Dessa forma, em 32 nós, o uso de 10 OSTs resulta em ganho de aproximadamente 23,6× sobre a configuração com 4 OSTs. Esse comportamento indica que, quando a concorrência entre processos aumenta, a configuração com menos OSTs passa a limitar a escalabilidade da E/S.

Assim, a análise conjunta das duas figuras indica que o aumento no número de OSTs melhora tanto a largura de banda agregada quanto o tempo médio das operações de envio nas classes de maior volume. A configuração com 10 OSTs permite melhor distribuição das requisições entre os alvos de armazenamento, enquanto a configuração com 4 OSTs concentra o tráfego em menos dispositivos e introduz gargalos mais visíveis no cenário com 32 nós.

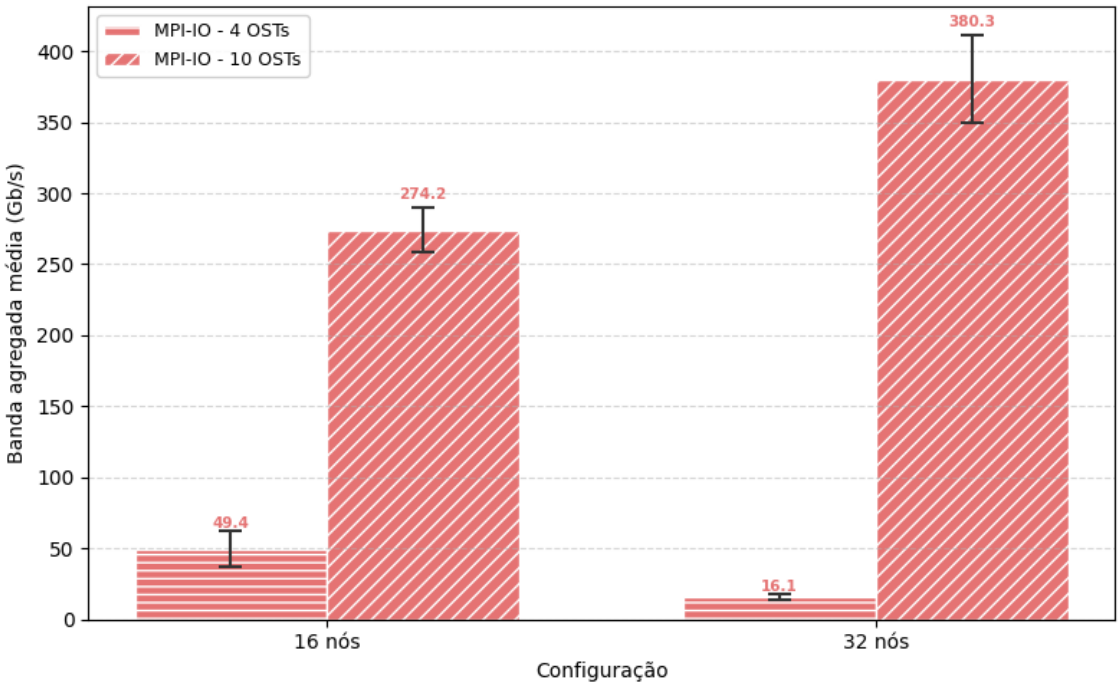


Figura 4.7: Banda agregada média percebida pela aplicação na implementação MPI-IO para configurações com 4 e 10 OSTs em 16 e 32 nós.

4.2 Experimento SDumont

O Experimento SDumont corresponde ao supercomputador Santos Dumont (SDumont), adquirido junto à empresa francesa EVIDEN/ATOS/BULL, que está localizado na sede do Laboratório Nacional de Computação Científica (LNCC), em Petrópolis-RJ, atuando como nó central (Tier-0) do Sistema Nacional de Processamento de Alto Desempenho (SINAPAD). Nesse experimento, este trabalho utilizou até 32 nós de processamento, cada um equipado com dois processadores Intel Xeon Cascade Lake Gold 6252, totalizando 24 núcleos físicos por nó, com suporte a *hyper-threading*, o que resulta em 48 processadores lógicos disponíveis. Cada nó conta ainda com 384 GB de memória e interconexão de alta velocidade baseada em InfiniBand EDR de 100 Gb/s, projetada para reduzir a latência e aumentar a eficiência em aplicações paralelas de larga escala. Nos experimentos, este trabalho utilizou a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre v2.12 implementado através do ClusterStor L300 da Cray/HPE. Essa implementação do Lustre reúne 1 nó MDS (com 1 MDT) e 6 nós OSSs (cada um servindo 1 OST). Este trabalho configurou o *stripe count* como 6, com *stripes* de 1 MB, disponibilizando um espaço total de armazenamento de 1,1 PB. Pelos motivos discutidos na introdução do capítulo, também aqui este trabalho configurou um *rank* por núcleo físico,

totalizando 24 *ranks* MPI por nó, que processam cenários em paralelo segundo o padrão *bag-of-tasks*.

Diferentemente do FSx for Lustre dedicado utilizado no Experimento AWS, o sistema de arquivos do SDumont é compartilhado com outras cargas de trabalho de HPC executando concorrentemente, o que introduz variabilidade adicional nas medições e amplifica o efeito da contenção sobre os recursos do sistema de arquivos nas maiores escalas. Esse compartilhamento faz do SDumont um cenário mais desafiador para a avaliação da escalabilidade de E/S do que o ambiente isolado da AWS.

4.2.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento SDumont. A Tabela 4.4 sumariza os resultados, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

O tempo total de execução medido decresce até 16 nós, mas perde eficiência rapidamente nas maiores escalas: 14.524 s em 2 nós, 7.758 s em 4 nós, 4.265 s em 8 nós, 3.130 s em 16 nós e 3.744 s em 32 nós. O speedup passa de 1,87× em 4 nós para 3,40× em 8 nós, 4,64× em 16 nós e recua para 3,88× em 32 nós, o que corresponde a queda de eficiência de 93,6% para 24,2%. Notavelmente, o tempo total piora entre 16 e 32 nós (3.130 s e 3.744 s, respectivamente), sintoma de que o gargalo da arquitetura centralizada já se manifesta a partir de 16 nós e passa a anular os ganhos de paralelismo. A banda agregada percebida confirma o diagnóstico: parte de 27,9 Gb/s em 2 nós, recua para 24,8 Gb/s em 4 nós e 23,9 Gb/s em 8 nós, colapsando a partir de 16 nós e mantendo-se entre 2,5 e 4,2 Gb/s no restante da faixa avaliada. A parcela de E/S, embutida no tempo de simulação, salta de 0,55% em 2 nós para 4,94% em 8 nós, 25,42% em 16 nós e 45,83% em 32 nós, evidência direta de que o gargalo da arquitetura centralizada se intensifica nas maiores escalas.

Tabela 4.4: Desempenho do Experimento SDumont com a implementação atual. O tempo total corresponde ao tempo de execução medido, que inclui computação, E/S e comunicação. Este trabalho calcula a eficiência em relação à base de 2 nós.

Nós	Banda percebida (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	$27{,}86 \pm 7{,}06$	$79{,}26 \pm 32{,}75$	0,55%	$383{,}25 \pm 28{,}01$	$14\,523{,}52 \pm 24{,}32$	1,00×	100,0%
4	$24{,}76 \pm 6{,}73$	$123{,}16 \pm 49{,}49$	1,59%	$370{,}35 \pm 20{,}95$	$7\,758{,}10 \pm 27{,}99$	1,87×	93,6%
8	$23{,}87 \pm 4{,}84$	$210{,}86 \pm 79{,}43$	4,94%	$383{,}38 \pm 22{,}87$	$4\,265{,}40 \pm 62{,}72$	3,40×	85,1%
16	$4{,}19 \pm 0{,}30$	$795{,}68 \pm 87{,}70$	25,42%	$389{,}72 \pm 6{,}26$	$3\,129{,}98 \pm 73{,}39$	4,64×	58,0%
32	$2{,}49 \pm 0{,}49$	$1\,715{,}81 \pm 259{,}79$	45,83%	$544{,}71 \pm 38{,}00$	$3\,743{,}71 \pm 249{,}11$	3,88×	24,2%

4.2.2 Avaliação da implementação MPI-IO com escritas independentes

Esta subseção analisa o desempenho da arquitetura proposta no Experimento SDumont, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre operado pelo ClusterStor L300. A Tabela 4.5 consolida os resultados.

Na implementação MPI-IO, o tempo total decresce ao longo de toda a faixa avaliada: 14.796 s em 2 nós, 7.800 s em 4 nós, 4.251 s em 8 nós, 2.519 s em 16 nós e 2.233 s em 32 nós. Em comparação com a implementação atual, a versão MPI-IO apresenta pequeno sobrecusto em 2 e 4 nós (aumento de 1,9% e 0,5%, respectivamente), mas passa a reduzir o tempo total a partir de 8 nós, com ganhos de 0,3%, 19,5% e 40,4% em 8, 16 e 32 nós. A banda agregada do *layer* de escrita, ao contrário do que ocorre na implementação atual, cresce de forma consistente com o paralelismo: de 5,5 Gb/s em 2 nós para 8,9 Gb/s em 4 nós, 12,1 Gb/s em 8 nós, 18,9 Gb/s em 16 nós e 34,1 Gb/s em 32 nós. O tempo médio de E/S por processo cai de 321 s em 2 nós para 64,1 s em 32 nós, e a parcela percentual de E/S no tempo de simulação permanece abaixo de 5% em todas as configurações.

Tabela 4.5: Desempenho do Experimento SDumont com a implementação MPI-IO. O tempo total corresponde ao tempo de execução medido, que inclui computação, E/S, comunicação e `MPI_File_open`/`MPI_File_close`. Este trabalho calcula a eficiência em relação à base de 2 nós.

Nós	Banda percebida (Gb/s)	I/O (s)	% I/O	Comunicação (s)	MPI_File_open/ MPI_File_close (s)	Total (s)	Speedup	Eficiência (%)
2	$5{,}55 \pm 0{,}37$	$321{,}24 \pm 85{,}21$	2,17%	$352{,}81 \pm 8{,}60$	$23{,}24 \pm 3{,}03$	$14\,796{,}47 \pm 46{,}32$	1,00×	100,0%
4	$8{,}89 \pm 0{,}97$	$271{,}52 \pm 50{,}04$	3,48%	$231{,}00 \pm 12{,}85$	$32{,}27 \pm 1{,}60$	$7\,800{,}23 \pm 93{,}32$	1,90×	94,8%
8	$12{,}09 \pm 1{,}51$	$194{,}71 \pm 34{,}40$	4,58%	$222{,}57 \pm 7{,}89$	$79{,}31 \pm 21{,}88$	$4\,251{,}01 \pm 141{,}68$	3,48×	87,0%
16	$18{,}85 \pm 1{,}19$	$100{,}92 \pm 21{,}10$	4,01%	$281{,}28 \pm 18{,}34$	$126{,}49 \pm 28{,}31$	$2\,519{,}43 \pm 100{,}75$	5,87×	73,4%
32	$34{,}10 \pm 5{,}36$	$64{,}06 \pm 26{,}99$	2,87%	$357{,}72 \pm 22{,}32$	$230{,}03 \pm 8{,}88$	$2\,232{,}82 \pm 86{,}57$	6,63×	41,4%

4.2.3 Análise comparativa

As figuras a seguir consolidam visualmente os resultados das duas subseções anteriores, apresentando, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações no Experimento SDumont.

A Tabela 4.6 resume os principais indicadores comparativos do Experimento SDumont. A coluna de melhoria no tempo total compara diretamente a implementação MPI-IO com a implementação atual para o mesmo número de nós, calculada como $(Total_{atual} - Total_{MPIIO})/Total_{atual}$. Valores positivos indicam redução do tempo total, enquanto valores negativos indicam piora. A MPI-IO apresenta sobre-

custo em pequena escala (–1,9% em 2 nós e –0,5% em 4 nós), mas passa a reduzir o tempo total a partir de 8 nós, com ganhos de 0,3%, 19,5% e 40,4% em 8, 16 e 32 nós. O ganho de banda também cresce nas maiores escalas: alcança 4,50× em 16 nós e 13,70× em 32 nós. A redução do tempo médio de E/S chega a 7,7% em 8 nós, 87,3% em 16 nós e 96,3% em 32 nós, atestando que a contenção do escritor único é o fator dominante de degradação da implementação atual nessas escalas.

Tabela 4.6: Resumo comparativo entre a implementação atual e MPI-IO no Experimento SDumont. Este trabalho calcula a melhoria no total e a melhoria de I/O em relação à implementação atual; valores negativos indicam piora.

Nós	Total atual (s)	Total MPI-IO (s)	Melhoria no total (%)	Banda percebida atual (Gb/s)	Banda percebida MPI-IO (Gb/s)	Ganho de banda perc.	I/O atual (s)	I/O MPI-IO (s)	Melhoria I/O (%)
2	14 523,52	14 796,47	−1,9%	27,86	5,55	0,20×	79,26	321,24	−305,3%
4	7 758,10	7 800,23	−0,5%	24,76	8,89	0,36×	123,16	271,52	−120,5%
8	4 265,40	4 251,01	0,3%	23,87	12,09	0,51×	210,86	194,71	7,7%
16	3 129,98	2 519,43	19,5%	4,19	18,85	4,50×	795,68	100,92	87,3%
32	3 743,71	2 232,82	40,4%	2,49	34,10	13,70×	1 715,81	64,06	96,3%

A Figura 4.8 compara a banda agregada média percebida pela aplicação obtida pela implementação atual e pela implementação MPI-IO. Em 2 e 4 nós, a implementação atual apresenta a maior banda observada, com 27,9 Gb/s e 24,8 Gb/s respectivamente, enquanto a MPI-IO atinge 5,5 Gb/s e 8,9 Gb/s — comportamento esperado em pequena escala, em que o paralelismo de escrita ainda não amortiza o custo fixo da camada MPI-IO. Em 8 nós, a implementação atual ainda registra 23,9 Gb/s, enquanto a MPI-IO atinge 12,1 Gb/s. A partir de 16 nós, o quadro se inverte: a implementação atual colapsa para 4,2 Gb/s, enquanto a MPI-IO continua crescendo e alcança 18,9 Gb/s — ganho de 4,50×. Em 32 nós, a diferença se acentua: a implementação atual cai para 2,5 Gb/s, enquanto a MPI-IO atinge 34,1 Gb/s, ganho de 13,70×.

Mais relevante que o valor absoluto é o contraste de *comportamento*: a implementação atual preserva banda elevada até 8 nós, mas exibe queda abrupta a partir de 16 nós e permanece em um patamar baixo nas escalas subsequentes (entre 2,5 e 4,2 Gb/s), sintoma da saturação do escritor central sob o Lustre compartilhado do SDumont. A implementação MPI-IO, em contrapartida, apresenta perfil monotonicamente crescente de banda, indicando que a infraestrutura de armazenamento ainda é capaz de absorver maior paralelismo de escrita.

Ainda assim, o desempenho absoluto da escrita paralela no SDumont permanece muito aquém do observado na AWS. O SDumont é um supercomputador **compartilhado por centenas de *jobs*** de diferentes usuários, que disputam simultaneamente os mesmos servidores de metadados (MDS) e OSTs do Lustre. Essa concorrência externa impõe um teto de banda: **mesmo com a MPI-IO**, a banda máxima al-

cançada (34,1 Gb/s em 32 nós) fica cerca de uma ordem de grandeza **abaixo** da obtida na AWS com o FSx for Lustre dedicado (380,3 Gb/s). Em outras palavras, a escrita paralela preserva a vantagem estrutural sobre o escritor único — eliminando o colapso de banda observado em larga escala —, mas seu desempenho absoluto no SDumont é limitado pela contenção com as demais cargas que compartilham o sistema de arquivos.

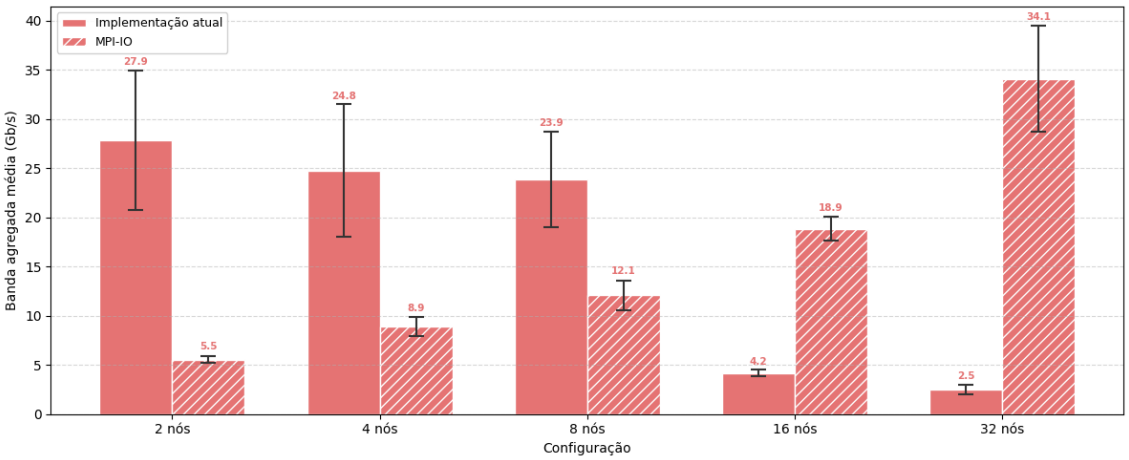


Figura 4.8: Banda agregada média percebida pela aplicação nas implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.9 detalha o tempo médio por processo no Experimento SDumont, separando as parcelas de computação, comunicação, E/S e `MPI_File_open`/`MPI_File_close`. A parcela `MPI_File_open`/`MPI_File_close` corresponde às chamadas de abertura e fechamento do arquivo de saída, executadas *uma única vez* por execução. Pelo mesmo motivo já discutido para o Experimento AWS, esse custo independe do tamanho da carga — não cresce com o número de cenários nem de estágios — e depende apenas do número de nós envolvidos na operação coletiva. As barras de erro indicam somente o desvio padrão do tempo de execução total. Em 2 e 4 nós, a implementação MPI-IO apresenta tempo total levemente superior ao da implementação atual: o custo das operações de coordenação e da própria camada MPI-IO ainda pesa nessas escalas, em que a contenção do escritor único da implementação atual ainda não saturou. A partir de 8 nós, a versão MPI-IO reduz o tempo total em relação à implementação atual, chegando a 40,4% de redução em 32 nós.

Na implementação atual, a parcela de E/S cresce de forma marcante a partir de 16 nós, chegando a representar uma fatia significativa do tempo de simulação. Esse comportamento é compatível com a saturação do processo escritor centralizado no Lustre compartilhado do SDumont. Na implementação MPI-IO, a aplicação redistribui o tempo de E/S entre os processos, que permanece reduzido em todas as

escalas (abaixo de 5% do tempo de simulação), enquanto o custo da `MPI_File_open`/`MPI_File_close` passa a compor uma fração visível do tempo total nas maiores configurações — em particular, cresce de cerca de 23 s em 2 nós para 126 s em 16 nós e 230 s em 32 nós.

Como já discutido para o Experimento AWS, a fração relativa da `MPI_File_open`/`MPI_File_close` observada nestes experimentos é maior do que a esperada em execuções reais do SDDP: com apenas três estágios mensais e 1 536 cenários, o tempo total de simulação é relativamente baixo, o que infla o peso percentual dessa parcela. Em execuções de operação programada típicas do SIN, com horizonte de vários anos e número de cenários muito maior, o custo absoluto de coordenação permanece o mesmo enquanto o tempo total de computação cresce em ordem de magnitude, diluindo essa fração para níveis desprezíveis. Assim, a figura evidencia que a abordagem proposta reduz o gargalo centralizado de escrita; o custo residual de coordenação, embora visível na escala dos experimentos, não é uma limitação intrínseca da arquitetura proposta.

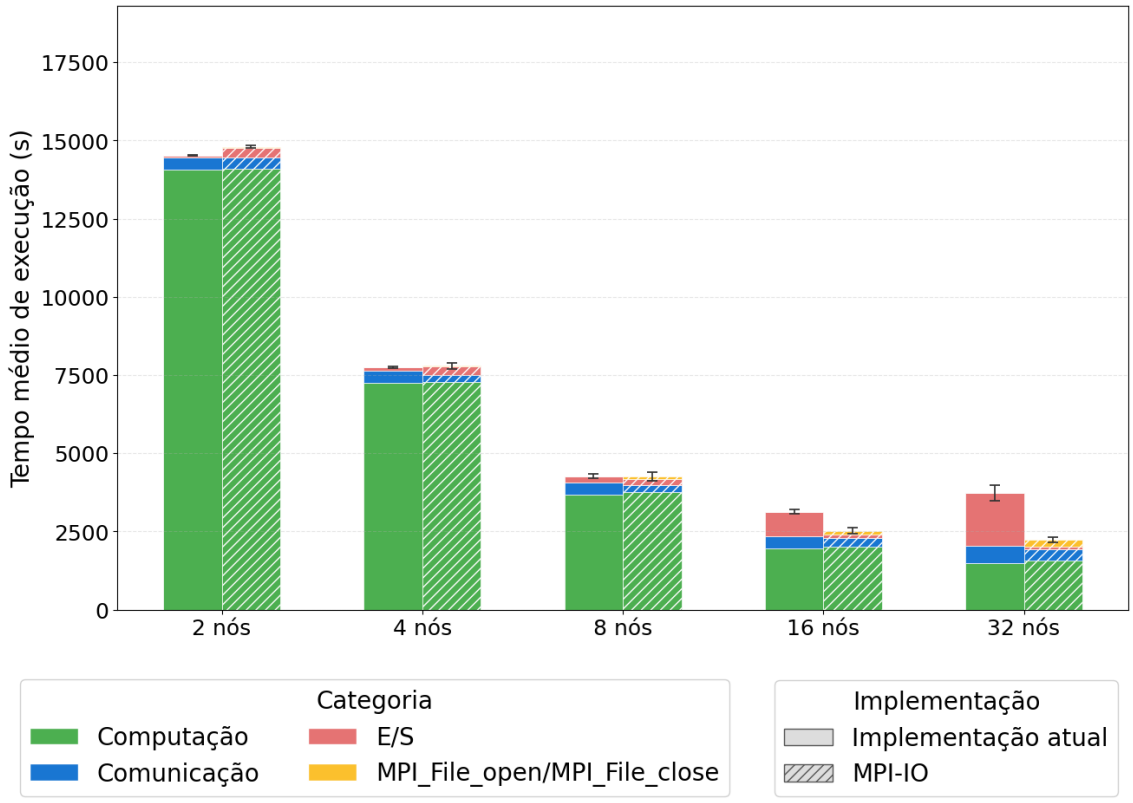


Figura 4.9: Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.10 apresenta o tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento SDumont, em escala

logarítmica, organizada em quatro painéis — um por categoria de tamanho.

O painel (a), correspondente a blocos de até 1 KB, mostra tempos de envio na ordem de poucas dezenas de microssegundos para ambas as implementações em todas as escalas, com diferenças marginais. Para essa granularidade, dominada por chamadas de controle e protocolo, nem o gargalo do escritor único nem o custo fixo da camada MPI-IO se manifestam de forma significativa.

O painel (b), referente a blocos de até 128 KB, evidencia o contraste mais dramático da figura. A implementação atual cresce de $1{,}4 \times 10^{-4}$ s em 2 nós para $2{,}7 \times 10^{-3}$ s em 8 nós e $5{,}3 \times 10^{-3}$ s em 32 nós, enquanto a implementação MPI-IO permanece estável em torno de $1{,}3 \times 10^{-5}$ s em todas as configurações. Nessa faixa, a contenção do processo escritor da arquitetura atual amplifica-se rapidamente com a concorrência, e a escrita paralela da MPI-IO chega a apresentar vantagem superior a duas ordens de grandeza em 32 nós.

Os painéis (c) e (d), referentes a blocos de até 1 MB e até 50 MB, mostram um comportamento misto. Em 2 e 4 nós, a implementação atual ainda apresenta tempos menores — 10,8 ms e 35 ms na classe de até 1 MB; 31 ms e 87 ms na classe de até 50 MB —, enquanto a MPI-IO paga seu custo fixo (45 ms, 71 ms, 123 ms e 250 ms, respectivamente). Essa vantagem inicial se inverte a partir de 8 nós: a saturação do escritor único na implementação atual eleva o tempo médio para 0,125 s na classe de até 1 MB e 0,254 s na classe de até 50 MB, contra 0,094 s e 0,433 s da MPI-IO. Em 16 nós, a vantagem da MPI-IO consolida-se: 0,082 s vs. 0,976 s na classe de até 1 MB (mais de uma ordem de grandeza) e 0,586 s vs. 1,60 s na classe de até 50 MB. Em 32 nós, a implementação atual volta a se degradar, chegando a 4,09 s na classe de até 1 MB e 8,08 s na classe de até 50 MB, enquanto a MPI-IO permanece em 0,095 s e 0,791 s, respectivamente.

Em conjunto, os painéis evidenciam um ponto de troca em função da escala: em 2 e 4 nós, a implementação atual é competitiva ou superior em blocos médios e grandes; em 8 nós, os resultados já favorecem a MPI-IO em parte das classes, mas a inversão torna-se dominante a partir de 16 nós, quando as classes de maior volume passam a sofrer forte contenção na implementação atual. Esse padrão também explica diretamente o colapso da banda agregada da implementação atual mostrado na Figura 4.8: a partir de 16 nós, a maior parte do volume escrito sofre saturação no escritor único.

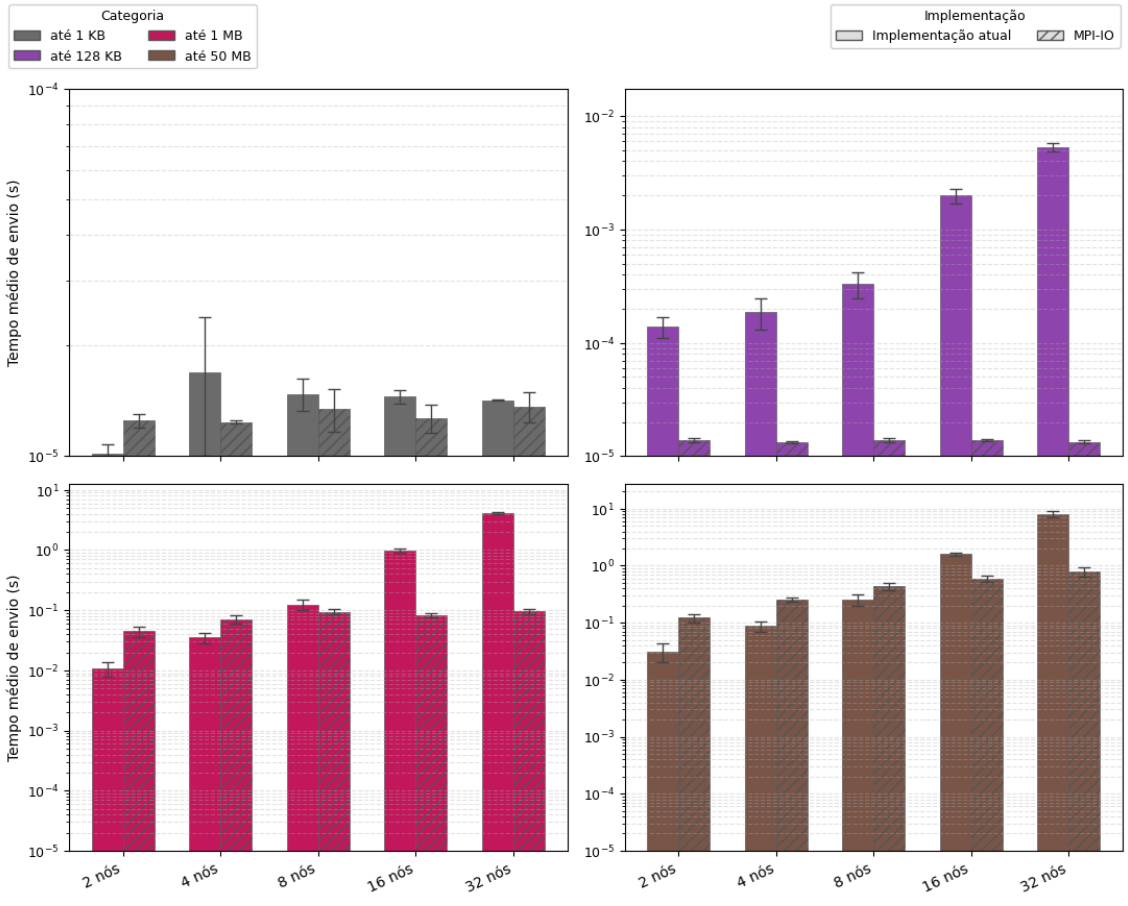


Figura 4.10: Tempo médio de bloqueio das operações de envio por tamanho de bloco de dados nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica).

Os resultados apresentados nesta seção demonstram que a adoção da abordagem MPI-IO com escritas independentes endereça o gargalo da arquitetura original associado à concentração das operações de E/S no processo escritor. No SDumont, esse benefício se manifesta a partir de 16 nós: a banda agregada cresce monotonicamente até 34,1 Gb/s em 32 nós, contra um perfil que colapsa e satura na implementação atual; a abordagem reduz o tempo de E/S em 87,3% e 96,3% em 16 e 32 nós, respectivamente. Em 2 e 4 nós, o custo fixo da camada MPI-IO ainda sobrepõe-se ao ganho de paralelismo de escrita, e a implementação atual é ligeiramente mais rápida. A partir de 8 nós, a MPI-IO também reduz o tempo total, com ganho crescente até 40,4% em 32 nós.

A queda de eficiência paralela observada em 32 nós — 24,2% na implementação atual e 41,4% na MPI-IO — e a variabilidade observada refletem, em boa medida, o caráter compartilhado do Lustre do SDumont, em que diferentes cargas de HPC concorrem simultaneamente pelo mesmo sistema de arquivos. Mesmo nesse cenário

hostil, o contraste qualitativo entre o perfil monotonicamente crescente de banda da MPI-IO e o colapso da implementação atual a partir de 16 nós persiste em toda a faixa avaliada, indicando que a vantagem estrutural da escrita paralela se mantém quando a infraestrutura impõe limites significativos.

4.3 Síntese das avaliações

Os dois experimentos — AWS, em ambiente de nuvem com FSx for Lustre dedicado, e SDumont, em supercomputador com Lustre compartilhado entre múltiplas cargas de HPC — convergem para um diagnóstico comum, ainda que com magnitudes diferentes. Em pequena escala (2 e 4 nós), a implementação atual é competitiva ou ligeiramente superior, pois o gargalo do escritor único ainda não saturou e o custo fixo da camada MPI-IO pesa em relação ao tempo total. A partir das maiores escalas, o quadro de E/S se inverte: na AWS esse efeito já aparece a partir de 8 nós, enquanto no SDumont o colapso da implementação atual se torna evidente a partir de 16 nós. Em ambos os casos, a degradação reflete a contenção sobre a serialização no nó escritor, enquanto a MPI-IO exibe perfil monotonicamente crescente de banda. Esse contraste qualitativo de comportamento — e não apenas o ganho pontual em uma ou outra escala — é o resultado mais robusto deste capítulo.

A magnitude do benefício, no entanto, depende fortemente da infraestrutura de armazenamento subjacente. No FSx for Lustre dedicado da AWS, a abordagem proposta atinge 380,3 Gb/s em 32 nós (ganho de 113× sobre a implementação atual) e reduz o tempo total em até 36,7%, com curva de banda ainda em crescimento. No Lustre compartilhado do SDumont, a banda da MPI-IO chega a 34,1 Gb/s em 32 nós (ganho de 13,7×), a redução do tempo de E/S chega a 96,3% e o tempo total cai até 40,4%. Esses ganhos ocorrem em condições adversas de concorrência por metadados e por banda do sistema de arquivos com outras aplicações em execução. A preservação da vantagem estrutural mesmo nesse regime sugere que a arquitetura proposta é portátil entre ambientes heterogêneos, e não dependente das condições idealizadas de uma plataforma dedicada.

A análise do impacto do número de OSTs na AWS reforça que o paralelismo de armazenamento subjacente é parte essencial do ganho: sair de 4 para 10 OSTs amplia a banda agregada em aproximadamente 5,6× em 16 nós e 23,6× em 32 nós, além de reduzir o tempo médio das escritas nas classes de bloco maiores. Essa observação sugere que, em deployments futuros, o dimensionamento da camada de armazenamento — número de OSTs e largura de *stripe* — deve constituir um parâmetro de projeto, e não um detalhe operacional.

Cabe, por fim, uma ressalva sobre a eficiência paralela absoluta. Apesar dos ganhos de E/S, a eficiência permanece bem abaixo do ideal linear nas maiores escalas,

situando-se em 43,5% na AWS e 41,4% no SDumont em 32 nós. Esse comportamento é esperado no regime de *strong scaling* adotado: com o problema de tamanho fixo, cada processo resolve um número cada vez menor de cenários à medida que se adicionam nós, de modo que custos aproximadamente constantes da fase de computação — sobretudo os associados ao acesso à memória — deixam de ser amortizados e passam a limitar o *speedup*. Trata-se de um gargalo de natureza computacional, distinto do gargalo de E/S tratado nesta dissertação; seu diagnóstico detalhado foge ao escopo deste trabalho e fica como tema de continuidade.

Esses resultados não esgotam, porém, o estudo da escalabilidade da fase de simulação final do SDDP. A faixa avaliada estendeu-se até 32 nós na AWS e no SDumont; configurações maiores podem revelar comportamentos adicionais, em particular saturação de metadados ou efeitos de coordenação coletiva. O custo da `MPI_File_open`/`MPI_File_close`, que aqui se mostrou desprezível em valor absoluto mas inflado em participação percentual pelo tamanho reduzido das execuções de teste, merece reavaliação em cargas reais de operação programada do SIN, com horizontes plurianuais. Por fim, o trabalho restringiu-se à fase de simulação final, em que cenários são mutuamente independentes; a aplicabilidade da estratégia à fase de convergência, com seu padrão diferente de comunicação e sincronização, fica como tema natural de continuidade.

Capítulo 5

Trabalhos relacionados

O trabalho de DICKENS e LOGAN [31] introduziu o uso de MPI-IO sobre o Lustre, identificando que as premissas de otimização do ROMIO conflitam com o modelo de byte-range locking do filesystem: para padrões não-interleaved, nos quais cada processo escreve em uma região exclusiva, escritas independentes evitam contenção de locks e superam as coletivas em desempenho. Esse diagnóstico abriu uma linha de pesquisa diretamente relevante para este trabalho, explorada em [32], [25], [33] e [34], e fundamenta a escolha de `MPI_File_write_at` como primitiva de E/S paralela no SDDP.

LIAO *et al.* [25] demonstraram que escritas independentes cujo tamanho não preenche uma unidade de *stripe* do Lustre forçam a aquisição de múltiplos locks sequenciais, degradando o desempenho. Como solução, os autores propuseram um mecanismo de *two-stage write-behind buffering* que acumula dados em memória até completar um bloco alinhado à fronteira de *stripe* antes de emitir a escrita ao filesystem. Esta dissertação *não adotou* essa abordagem: a implementação descrita no Capítulo 3 apenas agrega os registros horários por par (estágio, cenário) em uma única escrita, sem alinhamento explícito às fronteiras de *stripe*. Ainda assim, parte das escritas resultantes *acaba se alinhando incidentalmente* ao tamanho de *stripe* configurado no Lustre (1 MB nos experimentos), em decorrência do padrão de acesso do SDDP no formato horário: arquivos com algumas centenas de elementos do sistema elétrico produzem blocos por par (estágio, cenário) próximos ou múltiplos da fronteira de *stripe* (cf. Seção 3.3.2). Esse alinhamento incidental tende a beneficiar parte das operações de E/S do SDDP — evitando, para esses blocos, a aquisição de múltiplos locks descrita por LIAO *et al.* — e ajuda a explicar o desempenho já obtido pela arquitetura proposta mesmo na ausência de um mecanismo explícito de bufferização alinhada. A incorporação *sistemática* desse alinhamento permanece como direção de aprimoramento da arquitetura proposta, beneficiando inclusive os arquivos cujos blocos por cenário ficam desalinhados.

Em continuação direta, DICKENS e LOGAN [32] propuseram a Y-lib, uma bibli-

oteca que substitui o driver ADIO padrão do ROMIO para Lustre, implementando hierarchical striping e split writing para alinhar as operações de E/S às fronteiras de stripe e reduzir a contenção de locks. Os experimentos reportados pelos autores mostram ganhos de até 220% sobre a implementação padrão, confirmando que o Lustre requer tratamento específico para extrair desempenho em escritas paralelas.

GROPP *et al.* [33] formalizaram que, quando os acessos de diferentes processos não são interleaved — isto é, cada processo escreve em uma região exclusiva do arquivo —, a chamada `MPI_File_write_all` degenera para `MPI_File_write` acrescida de overhead de sincronização, tornando as escritas coletivas estritamente piores do que as independentes nesse regime. Esse resultado fundamenta teoricamente a escolha de `MPI_File_write_at` no SDDP, cujo padrão de acesso é precisamente não-interleaved.

LIAO e CHOUDHARY [34] mostraram que o particionamento de *file domains* do two-phase I/O do ROMIO não coincide com as fronteiras de *stripe* do Lustre, gerando contenção de locks mesmo em operações coletivas bem-formadas. Os autores propuseram um particionamento adaptativo ao protocolo de locking do filesystem subjacente, reduzindo essa contenção. Esse resultado complementa [31], reforçando que, para o padrão de acesso do SDDP, escritas independentes com offsets pré-calculados evitam inteiramente o problema de particionamento identificado pelos autores.

GARCIA-CARBALLEIRA *et al.* [35] apresentam o *Expand Ad-Hoc*, um sistema de arquivos paralelo *ad-hoc* baseado em servidores MPI em espaço de usuário que virtualiza o armazenamento local dos nós de computação — SSDs ou memória compartilhada — em um volume efêmero, cujo ciclo de vida coincide com o do *job* submetido: a aplicação escreve nesse volume durante a execução e a sincronização com o *backend* persistente (Lustre, GPFS) ocorre apenas nas etapas de *stage-in* e *stage-out*, fora da janela crítica da computação. Essa abordagem é diretamente relevante para a degradação observada no Experimento SDumont (Subseções 4.2.1 e 4.2.2), em que a concorrência entre o *job* do SDDP e outras cargas de HPC sobre o mesmo Lustre compartilhado impôs um teto absoluto de banda (34,1 Gb/s em 32 nós) uma ordem de grandeza inferior ao obtido na AWS com FSx for Lustre dedicado (380,3 Gb/s). Compor o MPI-IO com escritas independentes sobre uma camada como o Expand Ad-Hoc, instanciada nos próprios nós alocados ao *job* SDDP, contornaria inteiramente essa contenção externa, com migração ao Lustre apenas no *stage-out* final, e tem potencial de aproximar o desempenho observado no SDumont do desempenho obtido em um *backend* dedicado.

Capítulo 6

Conclusões e trabalhos futuros

Esta dissertação investigou os gargalos de E/S da fase de simulação final do algoritmo SDDP e propôs uma arquitetura paralela baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre, substituindo a estratégia atual com processo escritor único. A avaliação experimental conduzida em dois ambientes distintos — AWS (FSx for Lustre dedicado) e Santos Dumont (LNCC, Lustre compartilhado) — confirma que a arquitetura proposta endereça efetivamente o gargalo identificado e produz ganhos expressivos de desempenho e escalabilidade.

Na configuração de 32 nós, a arquitetura MPI-IO reduz o tempo total de execução em 36,7% no Experimento AWS e em 40,4% no Experimento SDumont, em relação à implementação atual. A banda agregada percebida pela aplicação cresce em 113× na AWS e 13,7× no SDumont. O contraste qualitativo entre as duas implementações é tão relevante quanto a magnitude desses ganhos: enquanto a implementação atual exibe colapso da banda a partir de 8 nós na AWS e de 16 nós no SDumont — decorrente da serialização inerente à implementação centralizada em um único processo escritor —, a implementação proposta apresenta perfil monotonicamente crescente de banda em toda a faixa avaliada, indicando que a infraestrutura de armazenamento ainda não atingiu seu limite nominal. Esse comportamento se sustenta em ambos os ambientes, atestando a portabilidade da solução entre infraestruturas heterogêneas — nuvem dedicada e supercomputador compartilhado.

A análise do impacto do número de *Object Storage Targets* (OSTs) reforça que o dimensionamento da camada de armazenamento é parte essencial do ganho: passar de 4 para 10 OSTs no FSx for Lustre amplia a banda agregada percebida em aproximadamente 5,6× em 16 nós e 23,6× em 32 nós, com redução do tempo médio das escritas nas classes de bloco maiores. Esse resultado sugere que, em *deployments* futuros, o `stripe_count` e o `stripe_size` do Lustre devem constituir parâmetros de projeto, e não um detalhe operacional.

Padrão de acesso dos arquivos diários. Dos 117 arquivos de resultado produzidos pelo SDDP nos experimentos, 104 seguem o formato horário e 13 seguem o formato diário. Embora minoritários em quantidade, os arquivos diários respondem por todas as escritas de blocos muito pequenos observadas no *workload*: cada registro corresponde a um dia do mês e, diferentemente dos arquivos horários — nos quais a aplicação agrega os registros horários de um par (estágio, cenário) em uma única escrita —, ela não agrupa os dias nos arquivos diários. A sobreposição entre E/S e computação proporcionada pelas escritas assíncronas mitiga parcialmente o impacto desse padrão fragmentado, mas a frequência de pequenas requisições ao Lustre permanece como fonte residual de sobrecarga e direciona uma das linhas de continuidade deste trabalho.

Disponibilidade de instâncias na nuvem. A execução do Experimento AWS enfrentou dificuldades de disponibilidade de instâncias `r7i.12xlarge` na mesma região, dado o caráter *memory-optimized* dessa família e a demanda concorrente sobre o *pool* de recursos. Essa limitação operacional restringiu a janela de experimentação, e o planejamento de futuras execuções em nuvem para cargas HPC deve considerá-la.

Trabalhos futuros

A partir dos resultados e limitações apresentados, este trabalho identifica as seguintes direções de continuidade:

- **Reorganização do padrão de acesso dos arquivos diários**, agrupando os registros mensais em uma única escrita por (estágio, cenário) — analogamente ao que a aplicação já faz nos arquivos horários —, de modo a eliminar a parcela residual de pequenas requisições ao Lustre identificada nestes experimentos.
- **Avaliação com adaptadores de rede de alta velocidade**, como InfiniBand HDR/NDR e *Elastic Fabric Adapter* (EFA) da AWS, para investigar em que medida a rede deixa de ser fator limitante e como o custo da Coordenação MPI-IO escala em redes de baixa latência.
- **Sensibilidade ao tamanho de *stripe* do Lustre**, com experimentos variando o `stripe_size` para mapear o *trade-off* entre fragmentação das requisições e distribuição efetiva entre os OSTs.
- **Buferização alinhada às *stripes* do Lustre**, substituindo o esquema atual — em que cada *rank* acumula em memória o bloco correspondente ao estágio completo de um cenário antes de submetê-lo em uma única chamada `MPI_File_iwrite_at` ou `MPI_File_write_at` — por uma política que segmente as

escritas pelos limites das *stripes*, de modo que cada submissão MPI-IO incida sobre exatamente uma *stripe* e, portanto, sobre exatamente um OST. Liao *et al.* [25] demonstram, em um contexto mais geral de escritas independentes em MPI-IO, que um esquema de bufferização em dois estágios alinhado ao particionamento do sistema de arquivos paralelo melhora significativamente o desempenho das escritas pequenas e desalinhadas; o mesmo princípio aplica-se diretamente ao perfil de E/S do SDDP horário, com potencial de reduzir a contenção entre *ranks* concorrentes sobre os mesmos OSTs.

- **Sensibilidade ao limite síncrono/assíncrono**, com experimentos variando o ponto de corte entre escritas assíncronas e síncronas (por exemplo, 64, 128 e 256 KB) para refinar empiricamente o parâmetro de projeto adotado neste trabalho.

- **Avaliação de *two-phase I/O***, isto é, de escritas coletivas com agregação interna pela biblioteca MPI-IO. Tentativas preliminares conduzidas durante esta pesquisa resultaram em *deadlock*, atribuído à combinação de múltiplas escritas por cenário com o desalinhamento do número de cenários processados por cada *rank* no padrão *bag-of-tasks*. A viabilização dessa estratégia exigiria mecanismos de sincronização que reconciliem a natureza dinâmica da distribuição de tarefas com a semântica coletiva das operações de E/S.

- **Diagnóstico das variações no tempo de computação para uma mesma configuração**, caracterizando as flutuações observadas na fase de computação entre execuções com o mesmo número de nós. Como discutido na Seção 4.3, a eficiência paralela permanece abaixo do ideal linear no regime de *strong scaling*: com o problema de tamanho fixo, custos aproximadamente constantes da computação — sobretudo os associados ao acesso à memória — deixam de ser amortizados à medida que se adicionam nós. A investigação detalhada desse gargalo computacional, distinto do gargalo de E/S tratado nesta dissertação (por exemplo, via *profiling* da hierarquia de memória e da largura de banda de acesso), permitiria isolar seus efeitos dos de E/S e orientar otimizações futuras da fase de simulação.

- **Investigação da piora pontual observada em 4 nós no Experimento AWS** (Seção 4.1.3), em que o tempo total da implementação MPI-IO superou o da implementação atual apesar do ganho já mensurável de banda. Esse resultado é indício de que fatores externos ao módulo de E/S — possivelmente variações no tempo de computação entre repetições — dominaram esse ponto específico da curva. Sua caracterização, articulada com o diagnóstico das

variações de computação mencionado acima, ajudaria a separar os efeitos de E/S dos efeitos computacionais na análise de escalabilidade.

- **Extensão da arquitetura à fase de geração de cortes do SDDP**, avaliando em que medida a estratégia de escritas independentes com MPI-IO se aplica ao padrão de comunicação e de sincronização dessa fase, distinto do da fase de simulação final tratada nesta dissertação.

A consolidação dessas direções pode contribuir para a construção de uma arquitetura de E/S ainda mais escalável e portátil, aplicável não apenas ao SDDP, mas a uma classe mais ampla de aplicações científicas com padrões *bag-of-tasks* e produção intensiva de resultados.

Referências Bibliográficas

[1] MESSAGE PASSING INTERFACE FORUM. "MPI: A Message-Passing Interface Standard, Version 4.1". 2023. `https://www.mpi-forum.org/`.

[2] ROSS, R., THAKUR, R., GROPP, W. *Parallel I/O for High Performance Computing*. Burlington, MA, Morgan Kaufmann, 2010.

[3] KANG, Q., LEE, S., HOU, K.-Y., et al. "Improving MPI Collective I/O for High Volume Non-Contiguous Requests With Intra-Node Aggregation", *IEEE Transactions on Parallel and Distributed Systems*, v. 31, n. 11, pp. 2682–2695, 2020. doi: 10.1109/TPDS.2020.3000458.

[4] LUU, H., WINSLETT, M., GROPP, W., et al. "A Multiplatform Study of I/O Behavior on Petascale Supercomputers". In: *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '15, pp. 33–44, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2749246.2749269.

[5] XIE, B., ORAL, S., ZIMMER, C., et al. "Characterizing Output Bottlenecks of a Production Supercomputer: Analysis and Implications", *ACM Transactions on Storage*, v. 15, n. 4, pp. 1–39, 2020. doi: 10.1145/3335205.

[6] MESSAGE PASSING INTERFACE FORUM. "MPI: A Message-Passing Interface Standard, Version 5.0". 2025. `https://www.mpi-forum.org/docs/`.

[7] OPENSFS AND EOFS. *Lustre Software Release 2.x: Operations Manual*, 2025. `https://doc.lustre.org/lustre_manual.pdf`.

[8] CONEJO, A. J., CARRIÓN, M., MORALES, J. M. *Decision Making Under Uncertainty in Electricity Markets*, v. 153, *International Series in Operations Research & Management Science*. New York, NY, Springer, 2010. ISBN: 9781441974211. doi: 10.1007/978-1-4419-7421-1.

[9] ZAKARIA, A., ISMAIL, F. B., LIPU, M. S. H., et al. "Uncertainty Models for Stochastic Optimization in Renewable Energy Applications", *Renewable Energy*, v. 145, pp. 1543–1571, 2020. doi: 10.1016/j.renene.2019.07.081.

[10] PEREIRA, M. V. F., PINTO, L. M. V. G. "Multi-stage Stochastic Optimization Applied to Energy Planning", *Mathematical Programming*, v. 52, n. 1–3, pp. 359–375, 1991.

[11] CIRNE, W., BRASILEIRO, F., SAUVÉ, J., et al. "Grid Computing for Bag of Tasks Applications". In: *Proceedings of the 3rd IFIP Conference on E-Commerce, E-Business and E-Government (I3E)*, 2003.

[12] MESSAGE PASSING INTERFACE FORUM. "MPI-2: Extensions to the Message-Passing Interface". 1997. Specification.

[13] GROPP, W., LUSK, E., SKJELLUM, A. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. 2 ed. Cambridge, MA, MIT Press, 1999. ISBN: 9780262571326.

[14] THAKUR, R., RABENSEIFNER, R., GROPP, W. "Optimization of Collective Communication Operations in MPICH", *The International Journal of High Performance Computing Applications*, v. 19, n. 1, pp. 49–66, 2005. doi: 10.1177/1094342005051521.

[15] THAKUR, R., GROPP, W., LUSK, E. "Data Sieving and Collective I/O in ROMIO". In: *Proceedings of the 7th Symposium on the Frontiers of Massively Parallel Computation*, pp. 182–189, 1999.

[16] BRAAM, P. J. "The Lustre Storage Architecture", *Cluster File Systems Inc.*, 2002.

[17] PETRINI, F., KERBYSON, D. J., PAKIN, S. "The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q". In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*, p. 55. ACM, 2003. doi: 10.1145/1048935.1050204.

[18] GRAHAM, R. L., SHIPMAN, G. M., BARRETT, B. W., et al. "Open MPI: A High-Performance, Heterogeneous MPI". In: *Proceedings of the 5th International Workshop on Algorithms, Models and Tools for Parallel Computing on Heterogeneous Networks*, 2005.

[19] JACKSON, K. R., RAMAKRISHNAN, L., MURIKI, K., et al. "Performance Analysis of High Performance Computing Applications on the Amazon Web Services Cloud". In: *Proceedings of the 2010 IEEE Second International Conference on Cloud Computing Technology and Science*, pp. 159–168. IEEE, 2010. doi: 10.1109/CloudCom.2010.69.

[20] TRAN, A., THEKKEDATH, B. "Running Simcenter STAR-CCM+ on AWS with AWS ParallelCluster, Elastic Fabric Adapter and Amazon FSx for Lustre". AWS Compute Blog, 2020. Disponível em: <https://aws.amazon.com/blogs/compute/running-simcenter-star-ccm-on-aws/>.

[21] AMAZON WEB SERVICES. "Elastic Network Adapter (ENA) — Enhanced Networking on Linux". Amazon EC2 User Guide, 2024. Disponível em: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-ena.html>.

[22] AMAZON WEB SERVICES. "Placement Groups for Amazon EC2 Instances". Amazon EC2 User Guide, 2024. Disponível em: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html>.

[23] OPERADOR NACIONAL DO SISTEMA ELÉTRICO. "Programa Mensal da Operação — PMO". https://www.ons.org.br/, 2023.

[24] ZHANG, Z., ESPINOSA, A., ISKRA, K., et al. "Design and Evaluation of a Collective IO Model for Loosely Coupled Petascale Programming". 2009. arXiv preprint.

[25] LIAO, W.-K., CHING, A., COLOMA, K., et al. "Improving MPI Independent Write Performance Using a Two-Stage Write-Behind Buffering Method". In: *Proceedings of the 21st IEEE International Parallel and Distributed Processing Symposium*, IPDPS '07. IEEE, 2007. doi: 10.1109/IPDPS.2007.370485.

[26] THAKUR, R., GROPP, W., LUSK, E. "On Implementing MPI-IO Portably and with High Performance", *Proceedings of the 6th Workshop on I/O in Parallel and Distributed Systems*, pp. 23–32, 1999.

[27] CARNS, P., LATHAM, R., ROSS, R., et al. "24/7 Characterization of Petascale I/O Workloads". In: *Proceedings of the 2009 IEEE International Conference on Cluster Computing and Workshops*, pp. 1–10. IEEE, 2009. doi: 10.1109/CLUSTR.2009.5289150.

[28] AMDAHL, G. M. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities". In: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), pp. 483–485, New York, NY, USA, 1967. Association for Computing Machinery. doi: 10.1145/1465482.1465560.

[29] INTEL CORPORATION. "Intel Xeon Scalable Processors — 4th Generation (Sapphire Rapids)". `https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html`, 2023.

[30] AMAZON WEB SERVICES. "Amazon EC2 R7i Instances — Memory Optimized". `https://aws.amazon.com/ec2/instance-types/r7i/`, 2024.

[31] DICKENS, P. M., LOGAN, J. "Towards an Understanding of the Performance of MPI-IO in Lustre File Systems". In: *Proceedings of the IEEE International Conference on Cluster Computing*, CLUSTER '08, pp. 254–263. IEEE, 2008. doi: 10.1109/CLUSTR.2008.4663791.

[32] DICKENS, P. M., LOGAN, J. "A High Performance Implementation of MPI-IO for a Lustre File System Environment", *Concurrency and Computation: Practice and Experience*, v. 22, n. 11, pp. 1433–1449, 2010. doi: 10.1002/cpe.1491.

[33] GROPP, W., KIMPE, D., ROSS, R., et al. "Self-Consistent MPI-IO Performance Requirements and Expectations". In: *Proceedings of the 15th European PVM/MPI Users' Group Meeting*, EuroPVM/MPI '08, pp. 167–176. Springer, 2008.

[34] LIAO, W.-K., CHOUDHARY, A. "Dynamically Adapting File Domain Partitioning Methods for Collective I/O Based on Underlying Parallel File System Locking Protocols". In: *Proceedings of the ACM/IEEE Conference on Supercomputing*, SC '08. IEEE, 2008. doi: 10.1109/SC.2008.5222722.

[35] GARCIA-CARBALLEIRA, F., CAMARMAS-ALONSO, D., CALDERON-MATEOS, A., et al. "A New Ad-Hoc Parallel File System for HPC Environments Based on the Expand Parallel File System". In: *Proceedings of the 22nd International Symposium on Parallel and Distributed Computing*, ISPDC '23, pp. 69–76. IEEE, 2023. doi: 10.1109/ISPDC59212.2023.00015.